

Calcolo Parallelo e Distribuito

Forme di Parallelismo – Classificazione di Flynn

Cenni sul calcolo distribuito

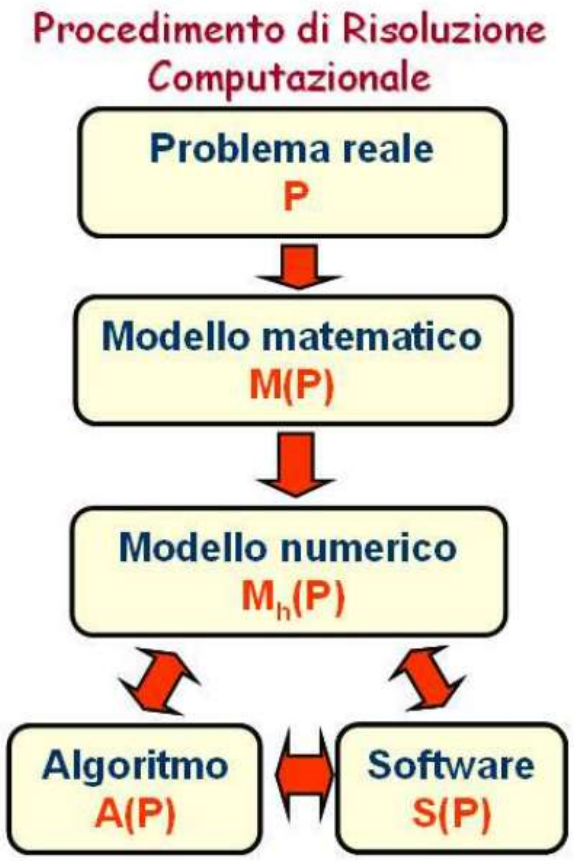
Docente: Prof. L. Marcellino

Tutor: Prof. P. De Luca

Modellizzazione di problemi su larga scala



- Ricerca su internet
- Trasporto
- Pubblicità e Marketing
- Servizi bancari e finanziari
- Media e intrattenimento
- Meteorologia
- Assistenza sanitaria
- Sicurezza informatica
- Formazione



Come ridurre il tempo di esecuzione di un software?

Negli ultimi anni la più grande sfida è stata ridurre questa quantità!



$$\tau = k \cdot T(N) \cdot \mu$$

T(N) = complessità computazionale dell'algoritmo

dipende dal metodo risolutivo scelto

μ = tempo d'esecuzione di un'opeazione f.p.

dipende dall'ambiente di calcolo

K è una costante moltiplicativa aleatoria che rappresenta ad esempio i tempi di I/O o i ritardi relativi alla scrittura in memoria

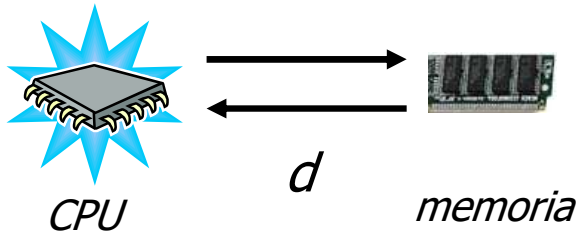
Come ridurre il tempo di esecuzione di un software?

Negli ultimi anni la più grande sfida è stata ridurre questa quantità!



$$\tau = k \cdot T(N) \cdot \mu$$

riducendo μ
migliorando la tecnologia
(miniaturizzazione)



problemi di surriscaldamento!!!

limiti tecnologici

Come ridurre il tempo di esecuzione di un software?

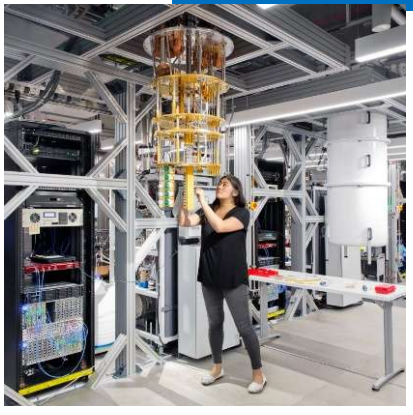
Negli ultimi anni la più grande sfida è stata ridurre questa quantità!



$$\tau = k \cdot T(N) \cdot \mu$$

riducendo K
modificando la tecnologia
(quantum computing)

computer con una diversa rappresentazione numerica nella memorizzazione dei dati (accesso più rapido alla memoria).
Algoritmi quantistici: importanti cambiamenti concettuali degli algoritmi standard.



ancora in fase embrionale

Come ridurre il tempo di esecuzione di un software?

Negli ultimi anni la più grande sfida è stata ridurre questa quantità!



$$\tau = k \cdot T(N) \cdot \mu$$

riducendo T(N)
ottimizzando l'algoritmo

risoluzione di un sistema diagonale

$$\begin{pmatrix} a_1 & 0 & \cdots & 0 \\ 0 & a_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & \cdots & \cdots & a_n \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ \vdots \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} y_1 \\ \vdots \\ \vdots \\ y_n \end{pmatrix}$$

Algoritmo "immediato"

$x_i = y_i / a_i \quad i = 1,..,n$

T(n)=n

Non esiste un algoritmo con complessità inferiore

E' possibile dimostrare
(teoria della complessità degli algoritmi)
che per alcune classi di problemi
esistono algoritmi con complessità computazionale minima
(**algoritmi ottimali**)

limiti teorici

Come ridurre il tempo di esecuzione di un software?

Negli ultimi anni la più grande sfida è stata ridurre questa quantità!



$$\tau = k \cdot T_{np}(N) \cdot \mu$$

Calcolo Parallelo usando np unità processanti

Calcolo Parallelo

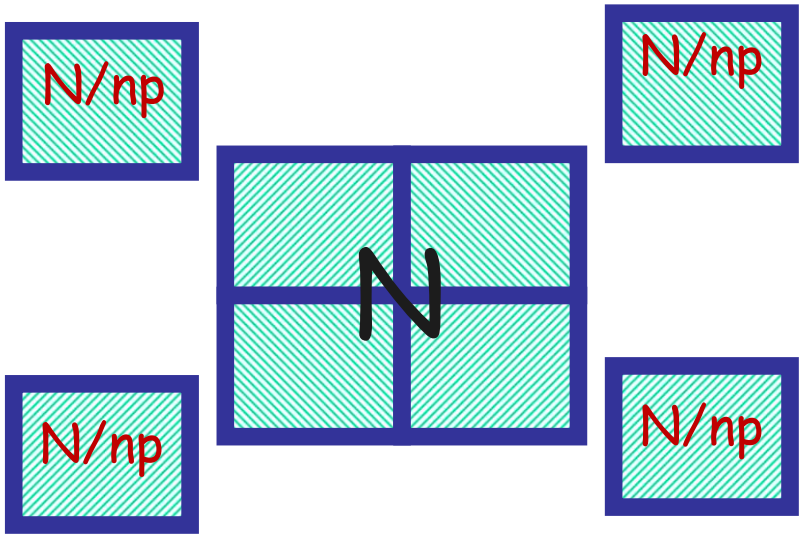
Il **calcolo parallelo** è una evoluzione del calcolo seriale che tenta di emulare ciò che spesso avviene nel mondo naturale: molteplici eventi complessi e inter-correlati che avvengono nello stesso tempo.



L'idea del **calcolo parallelo** si basa sull'impiego simultaneo di risorse di calcolo multiple per risolvere un unico problema, spezzandolo in parti discrete elaborabili contemporaneamente, ovvero che possono essere eseguite in modo seriale su **differenti CPU**.

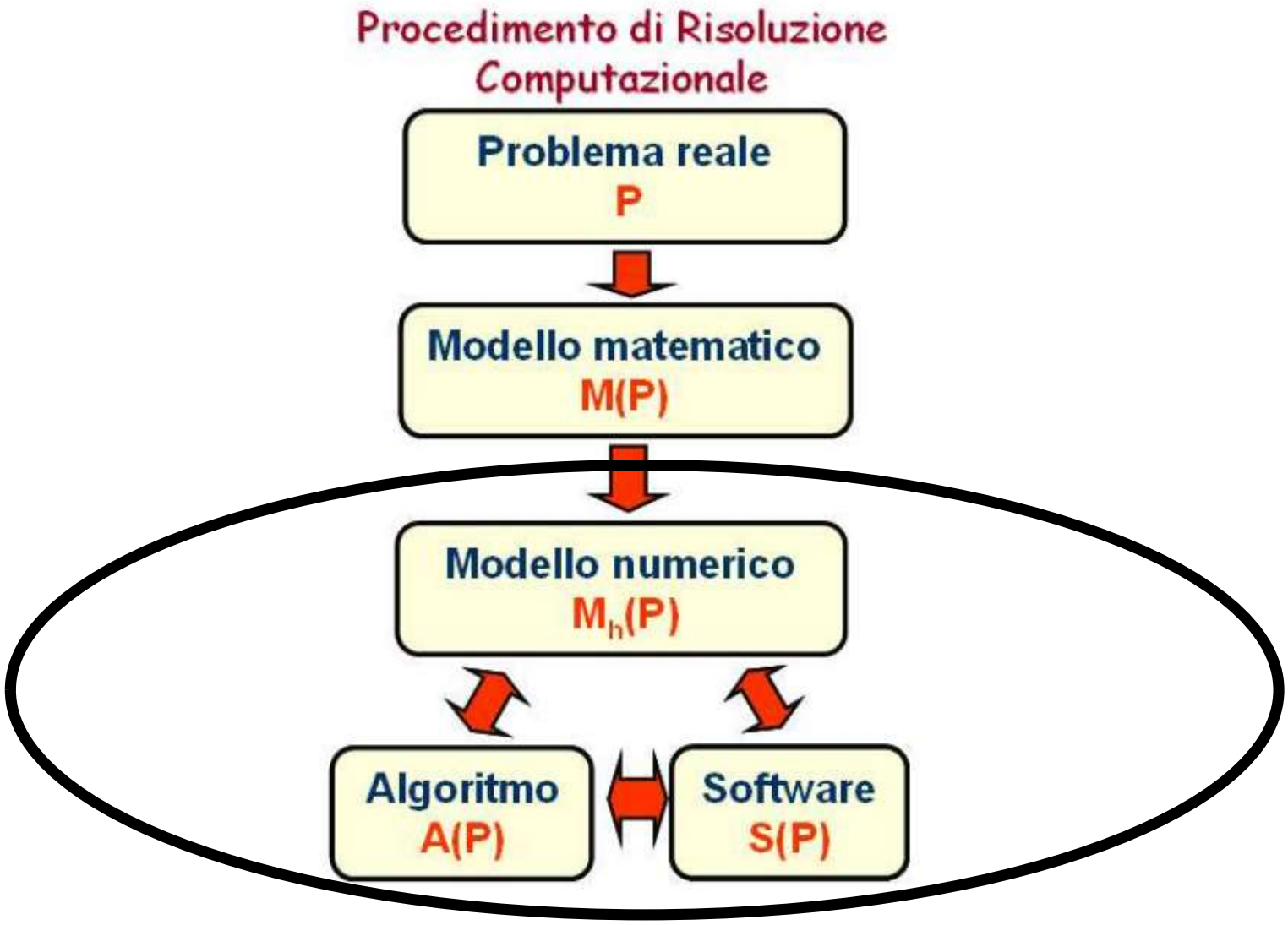
Calcolo Parallelo

Decomporre un problema di dimensione N
in **np** sottoproblemi di dimensione N/**np**
e risolverli contemporaneamente
usando **np** CPU

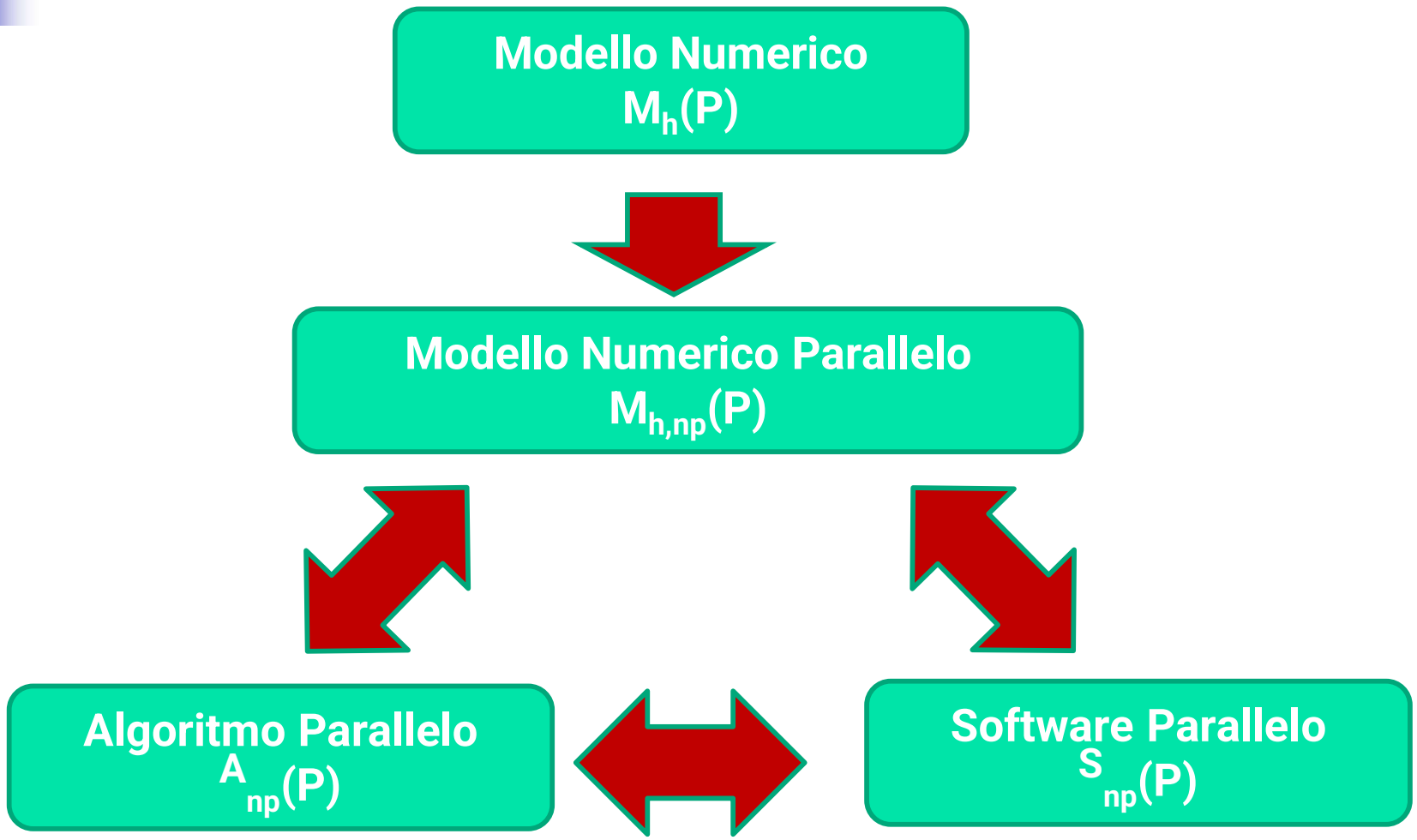


Sviluppo di appositi strumenti computazionali
hardware – software - algoritmi

Modellizzazione di problemi su larga scala



algoritmi e software paralleli



Anche in questo caso, per progettare un algoritmo parallelo, è fondamentale non perdere mai di vista quali siano le capacità intrinseche dell'architettura parallela a disposizione o necessaria.

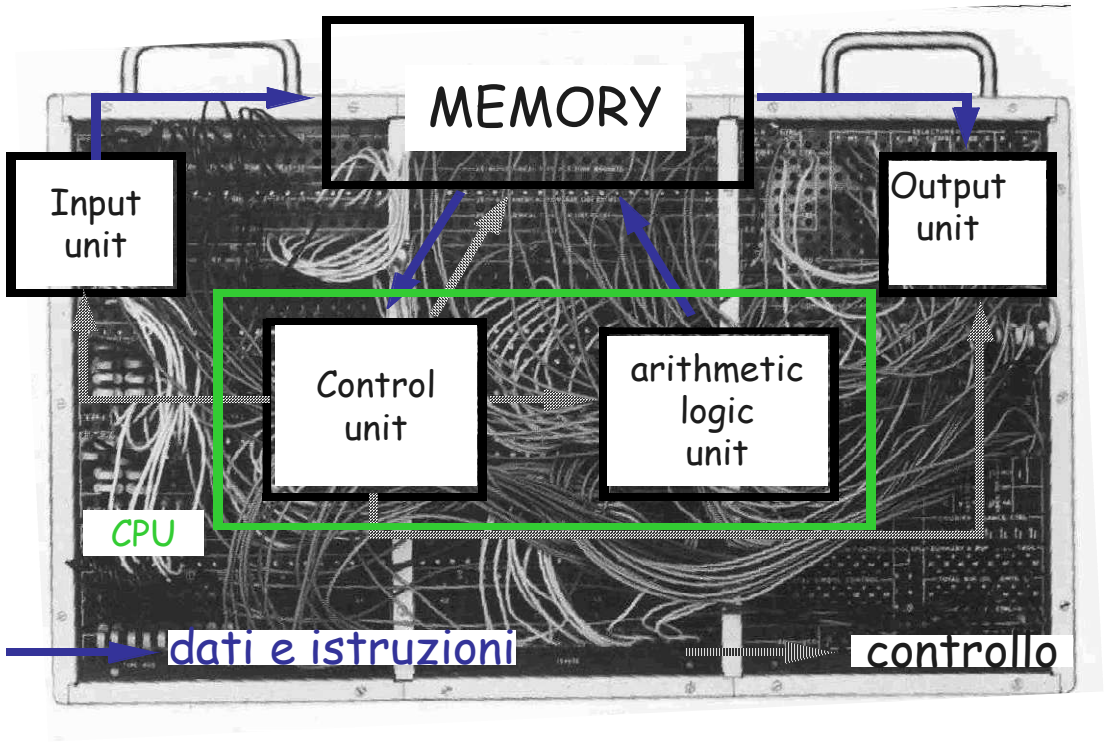
Hardware per il Calcolo Parallelo

Per approcciarsi al calcolo parallelo è necessario, innanzitutto, avere una visione chiara di come è fatto l'hardware delle moderne architetture di calcolo avanzato.



Questa è la macchina che tutti conosciamo

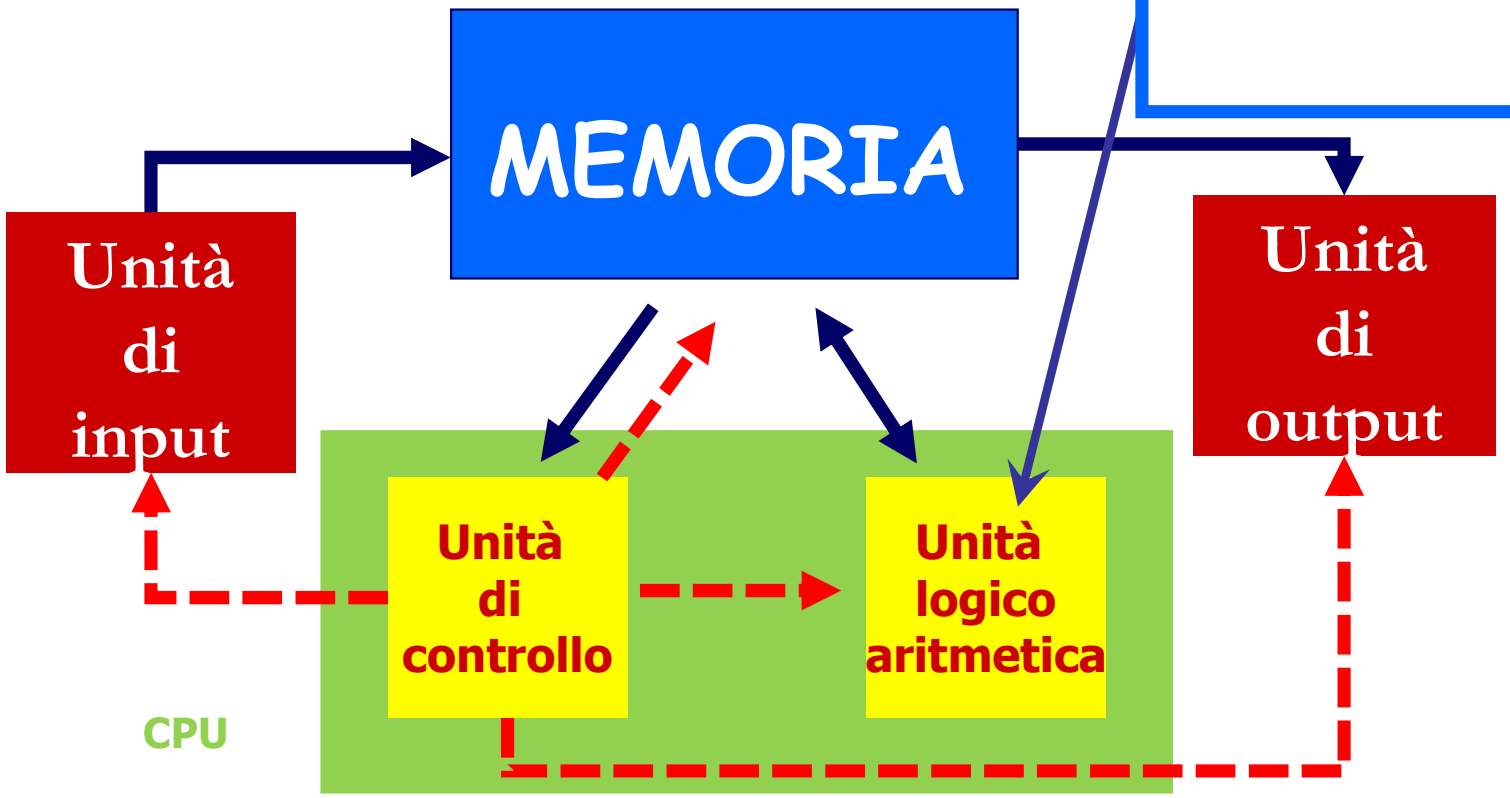
1945 John Von Neumann



parallelismo temporale

Macchina di Von Neumann

più unità funzionali all'interno di un'unica ALU

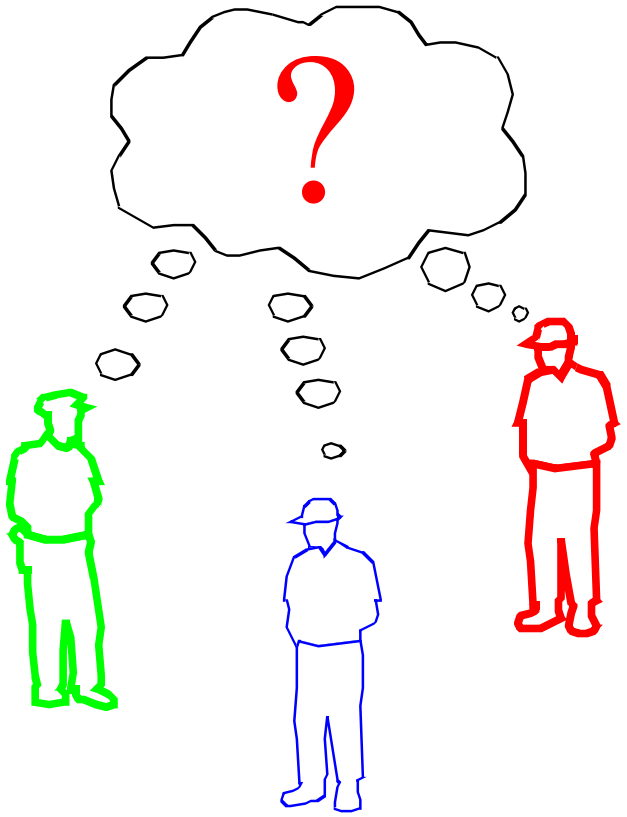


Dati e Istruzioni — controllo

Primo tipo di parallelismo *on-chip*

Esempio: costruzione di una casa

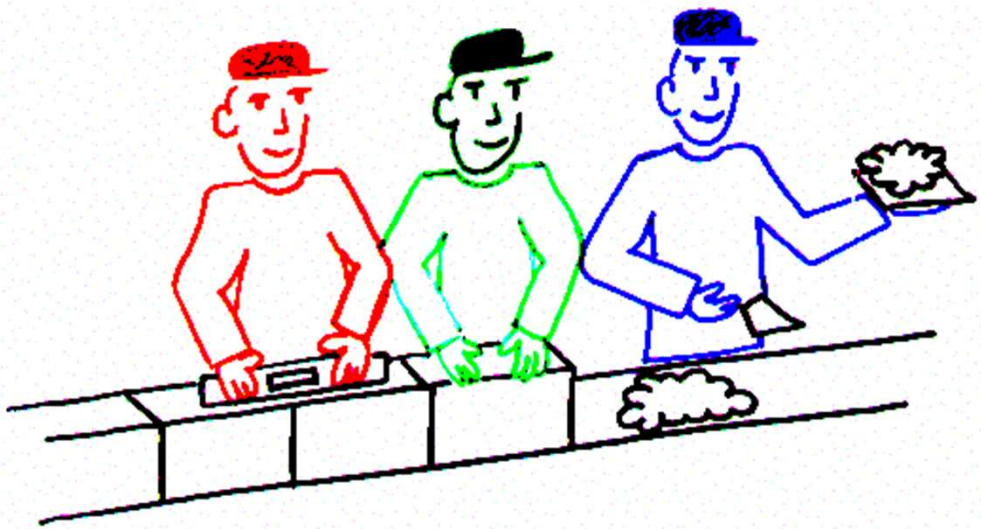




Primo tipo di parallelismo

Primo tipo di parallelismo

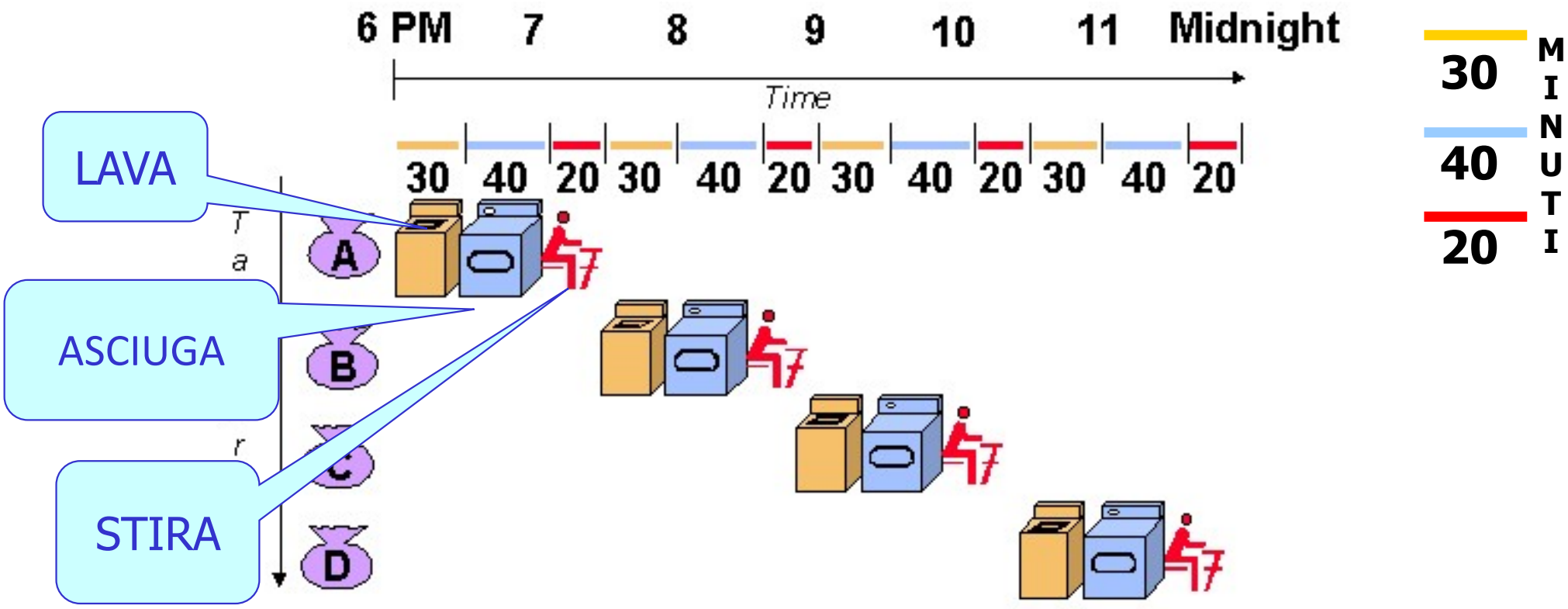
Tecnica della catena di montaggio (pipeline)



I tre operai eseguono contemporaneamente fasi successive dello stesso lavoro

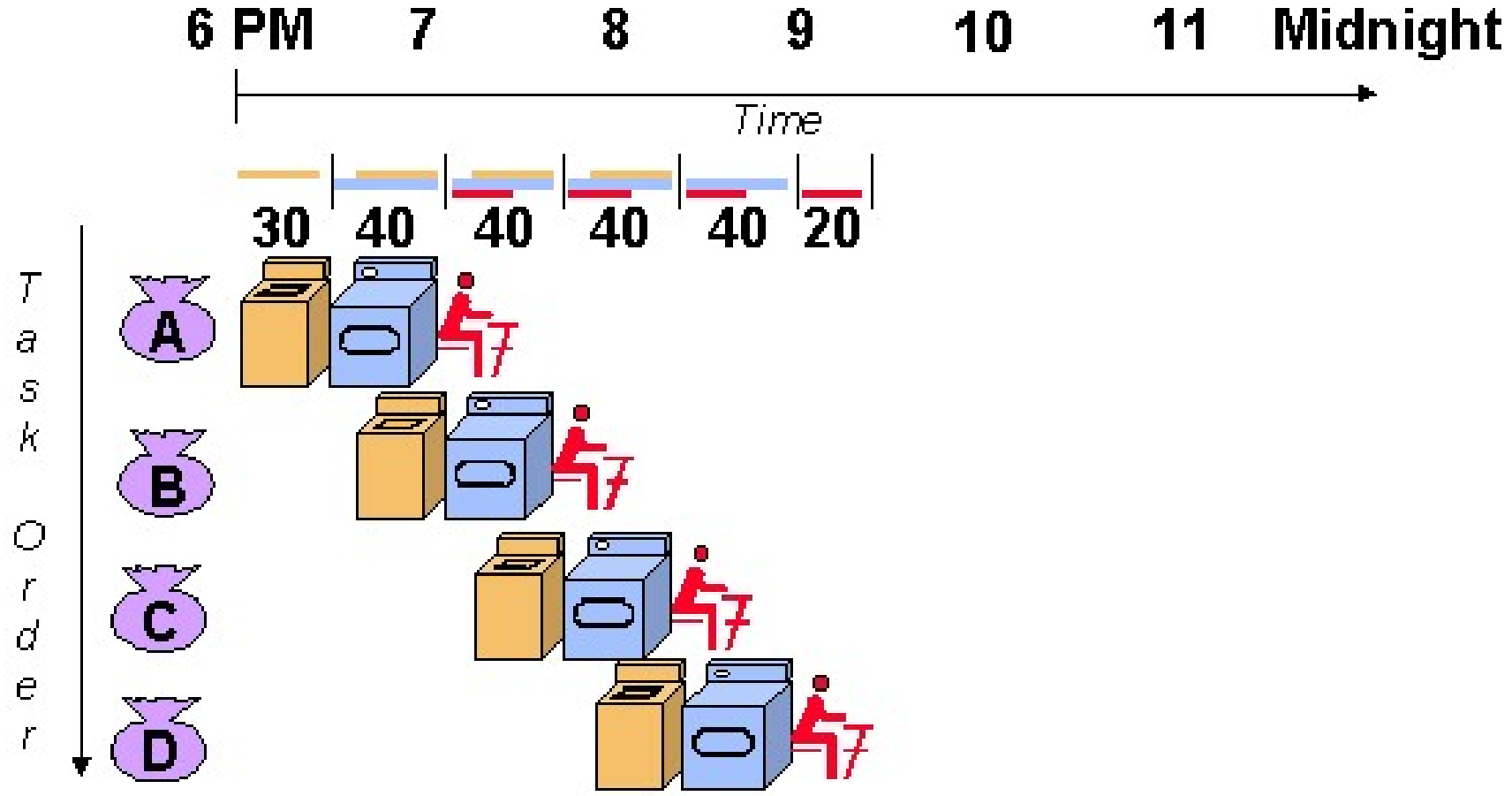
PARALLELISMO TEMPORALE

Esempio



Per completare il lavoro sono necessarie 6 ore!

Quanto tempo si risparmia?

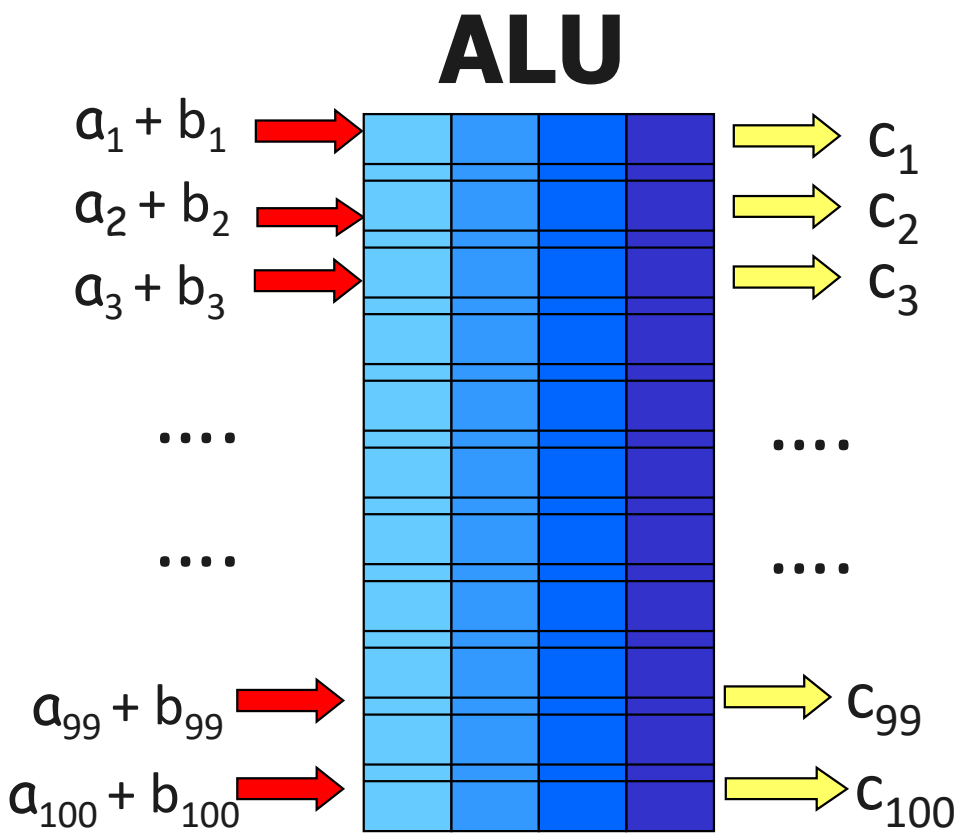


Con una pipeline bastano 3 ore !

Esempio: somma di 100 numeri floating point

$$c_i = a_i + b_i \qquad i=1,100$$

Unità tradizionale



Esempio: somma di 100 numeri floating point

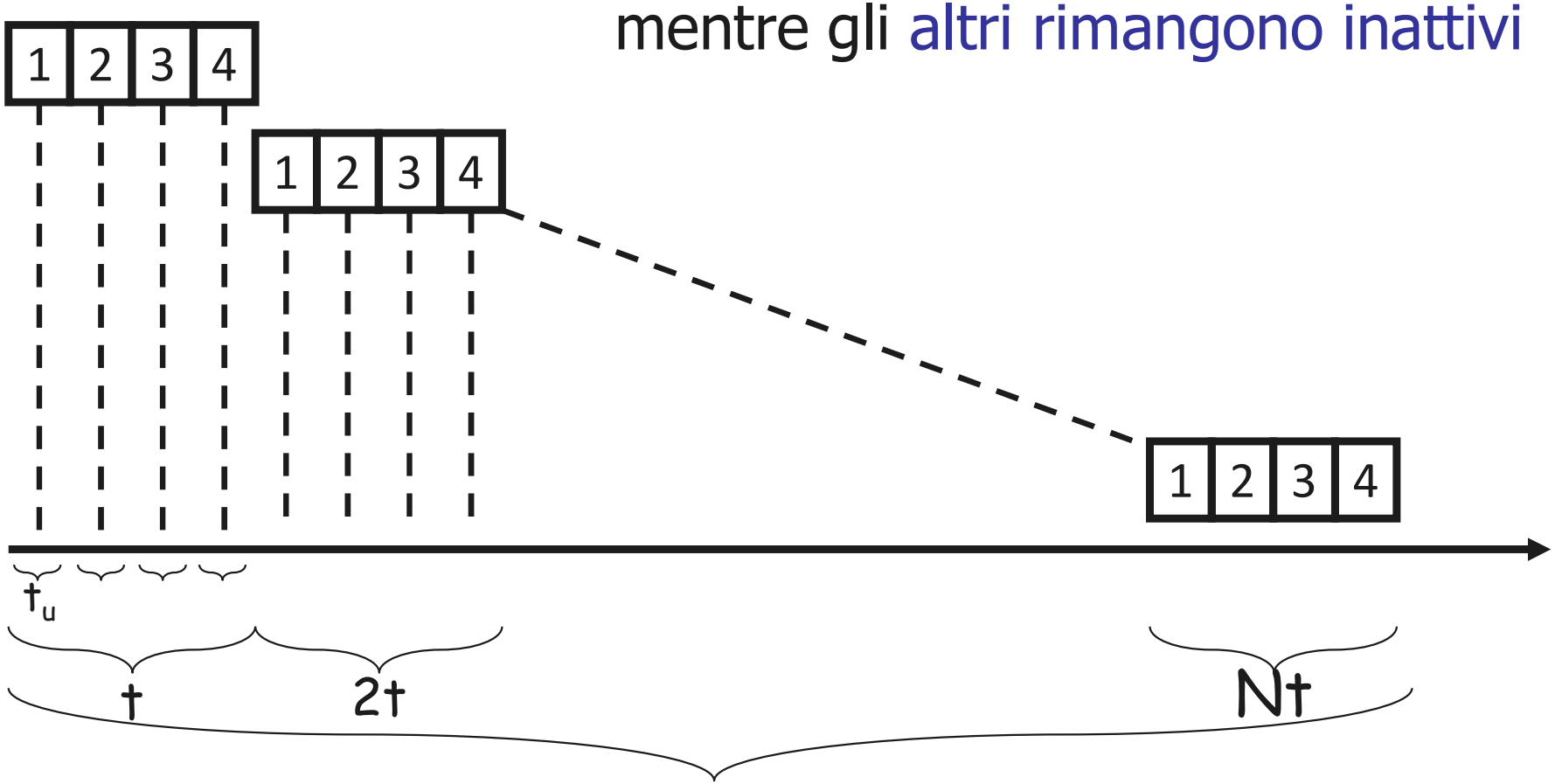
L'unità funzionale per l'addizione floating point è divisa in **4** segmenti

Ciascun segmento è preposto alla esecuzione di una fase dell'operazione, ad esempio
Per la somma:

CONFRONTO DEGLI ESPONENTI	FASE 1
SHIFT DELLA MANTISSA	FASE 2
SOMMA DELLE MANTISSE	FASE 3
NORMALIZZAZIONE	FASE 4

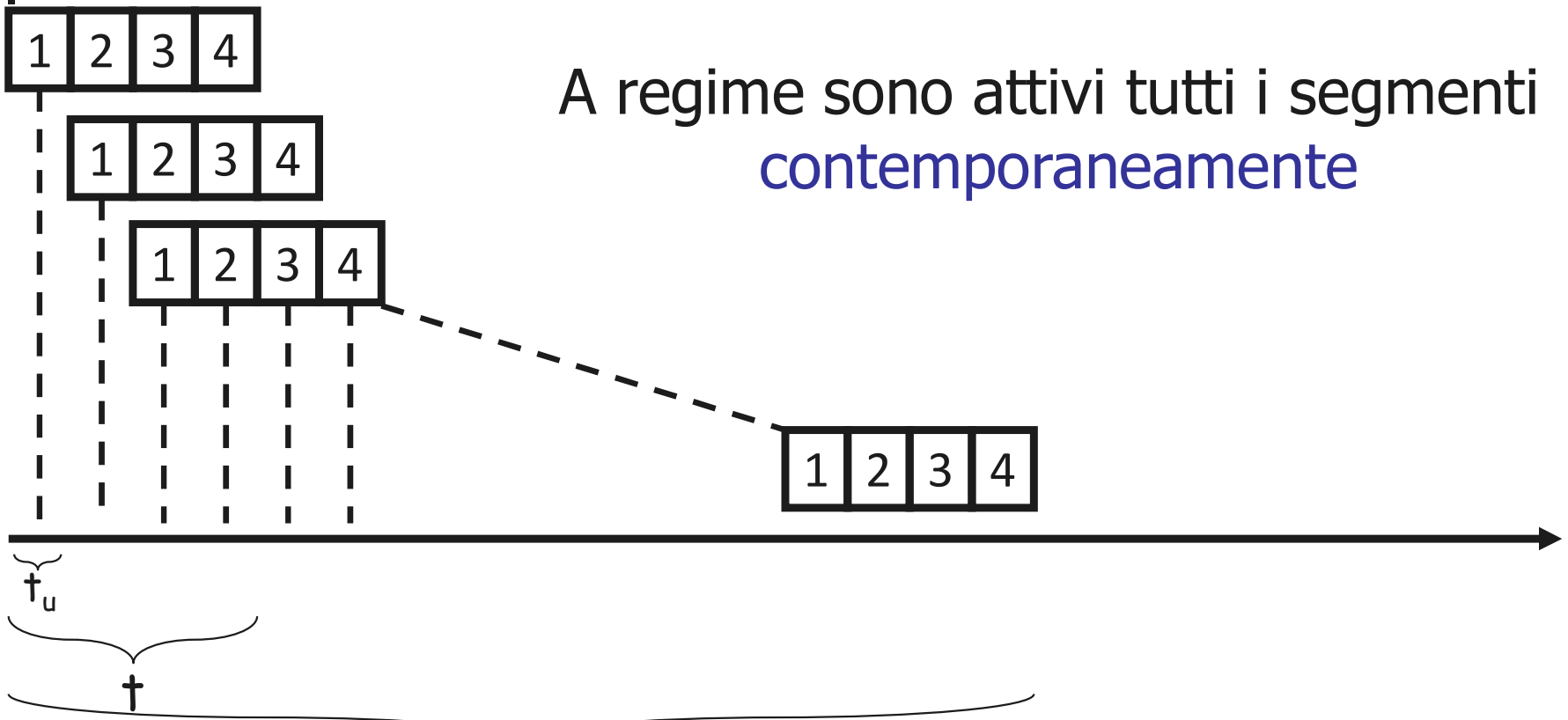
Esecuzione tradizionale di N somme f.p.

Solo un segmento alla volta è attivo
mentre gli altri rimangono inattivi



$$T = Nt = 4Nt_u$$

Esecuzione *pipelined* di N somme f.p.



A regime sono attivi tutti i segmenti contemporaneamente

$$T = [4+(N-1)]t_u < 4\ N\ t_u$$

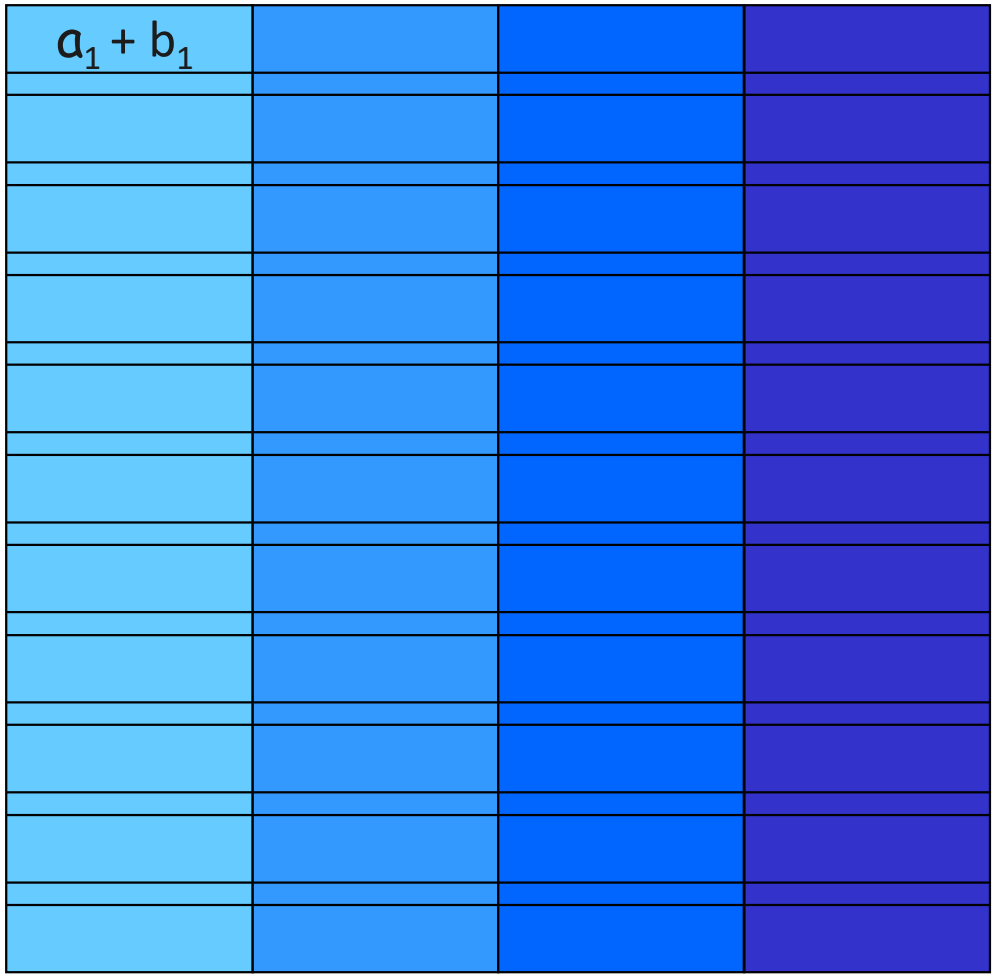
Numero di "segmenti" del pipe

Esempio: somma di 100 numeri floating point

$c_i = a_i + b_i \qquad i=1,100$

Unità pipelined

ALU



Hardware:

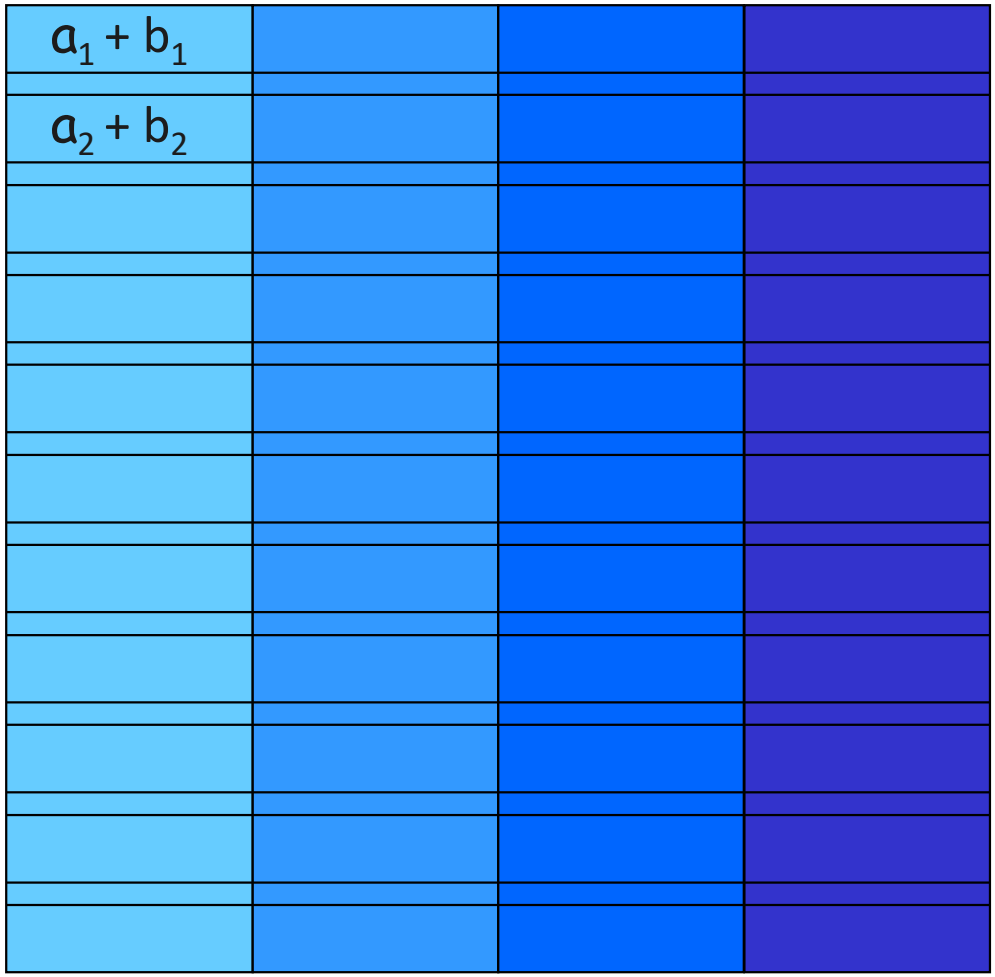
più unità funzionali all'interno di un'unica ALU

Esempio: somma di 100 numeri floating point

$c_i = a_i + b_i \qquad i=1,100$

Unità pipelined

ALU



Hardware:

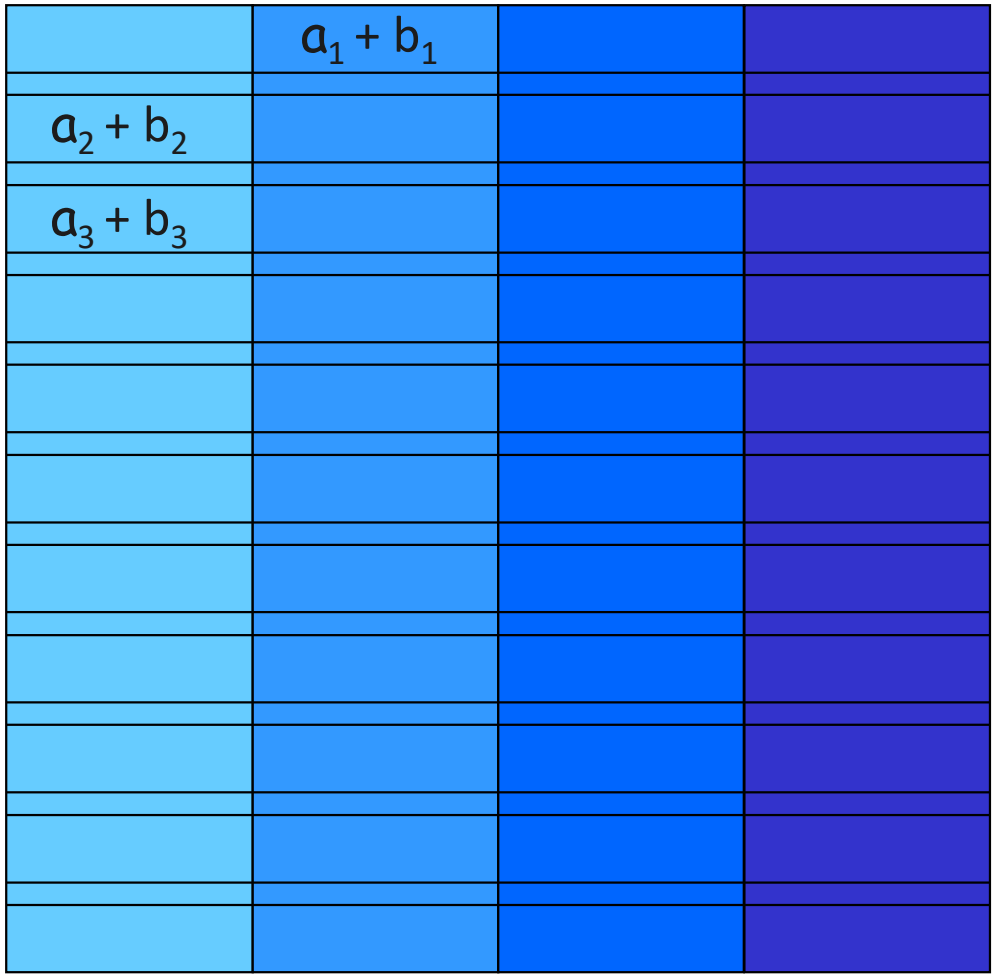
più unità funzionali all'interno di un'unica ALU

Esempio: somma di 100 numeri floating point

$c_i = a_i + b_i \qquad i=1,100$

Unità pipelined

ALU



Hardware:

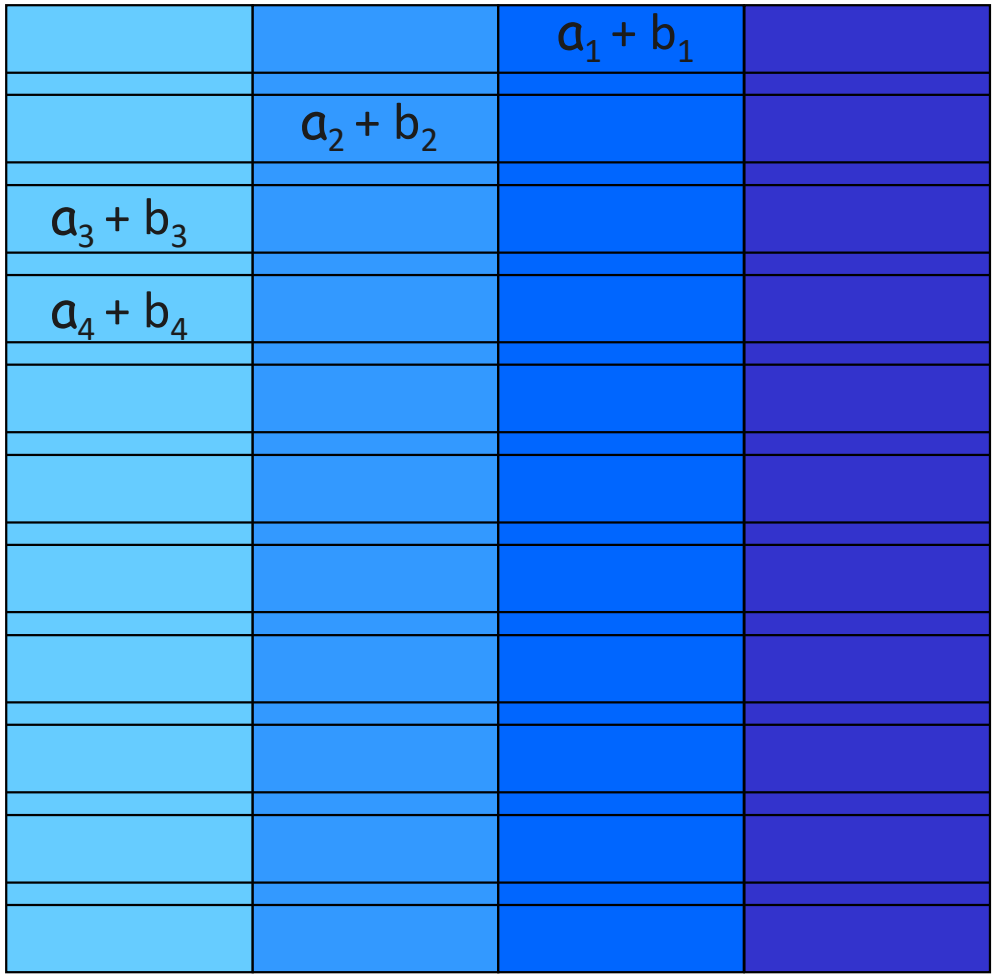
più unità funzionali all'interno di un'unica ALU

Esempio: somma di 100 numeri floating point

$c_i = a_i + b_i \qquad i=1,100$

Unità pipelined

ALU



Hardware:

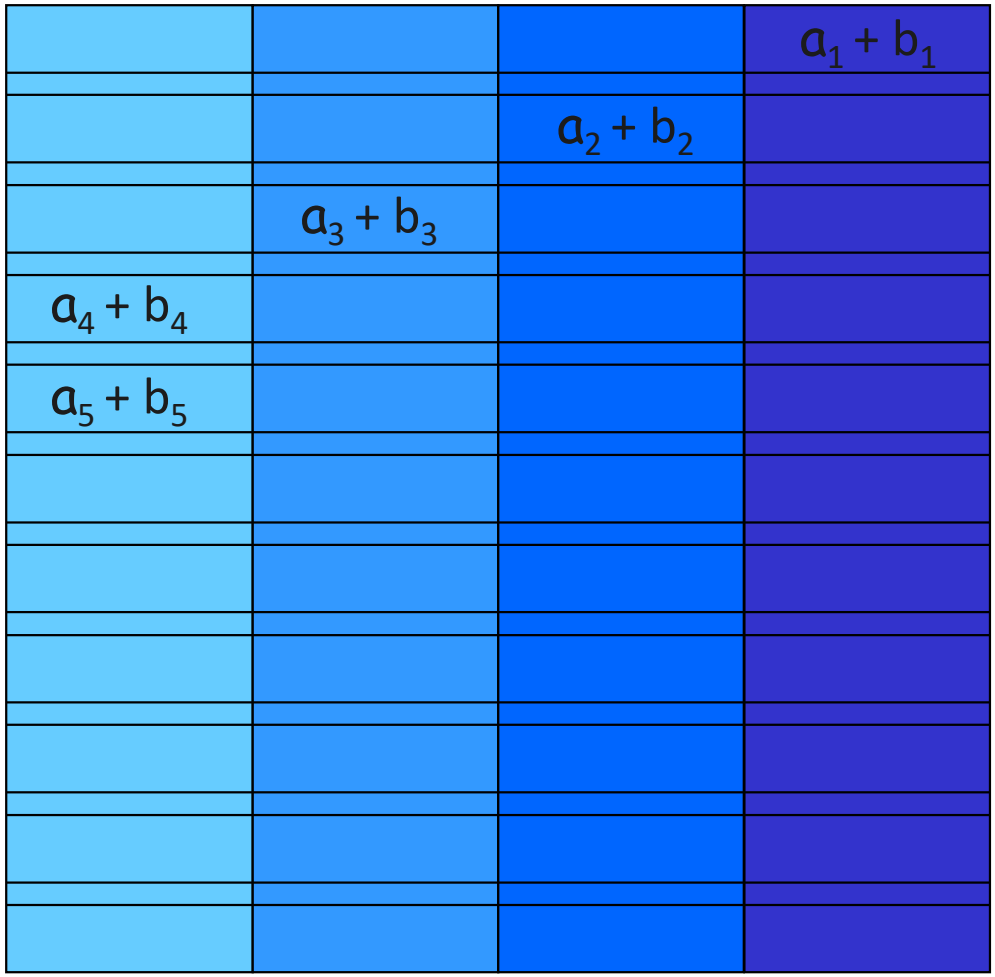
più unità funzionali all'interno di un'unica ALU

Esempio: somma di 100 numeri floating point

$c_i = a_i + b_i \qquad i=1,100$

Unità pipelined

ALU



Hardware:

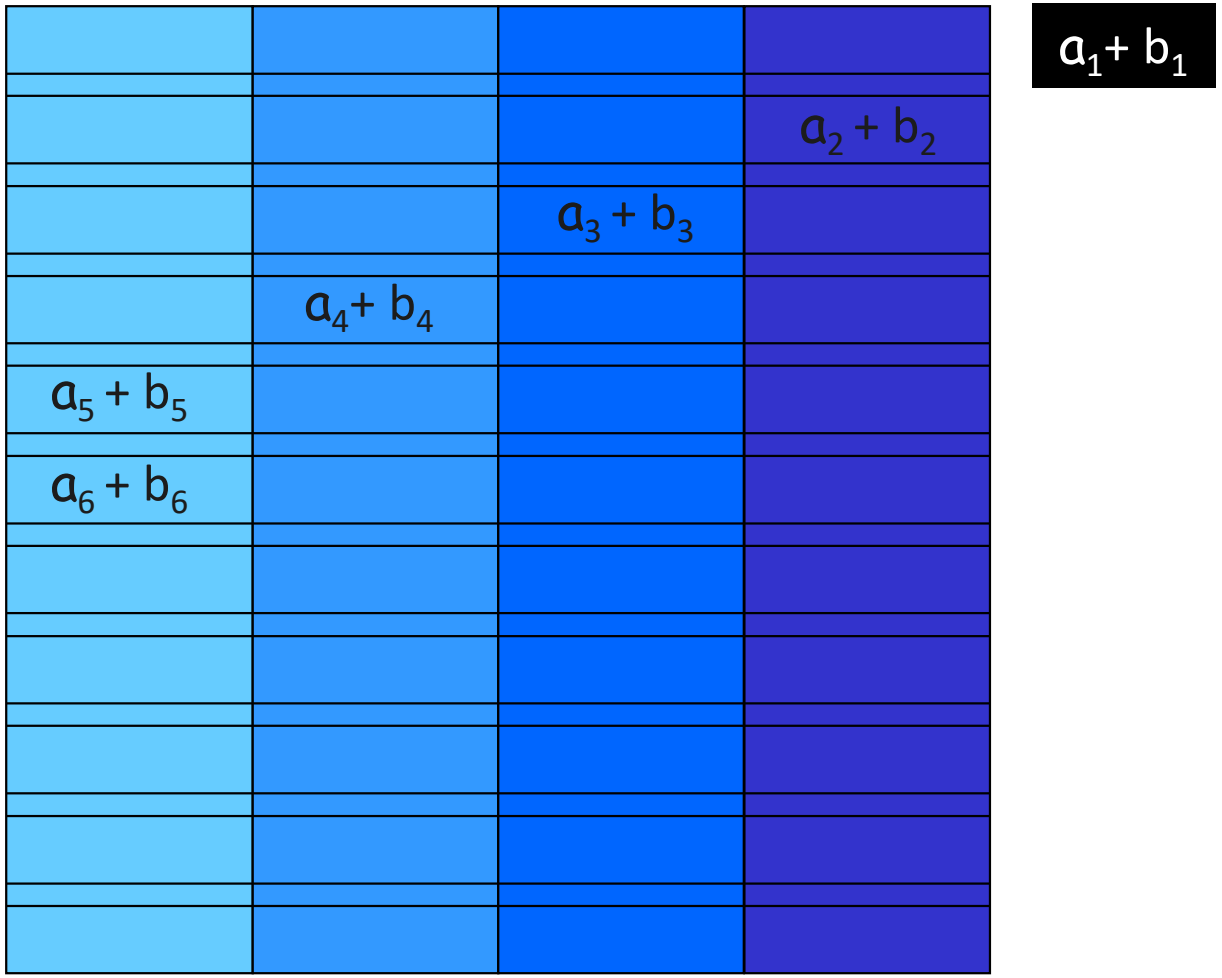
più unità funzionali all'interno di un'unica ALU

Esempio: somma di 100 numeri floating point

$c_i = a_i + b_i \qquad i=1,100$

Unità pipelined

ALU



Hardware:

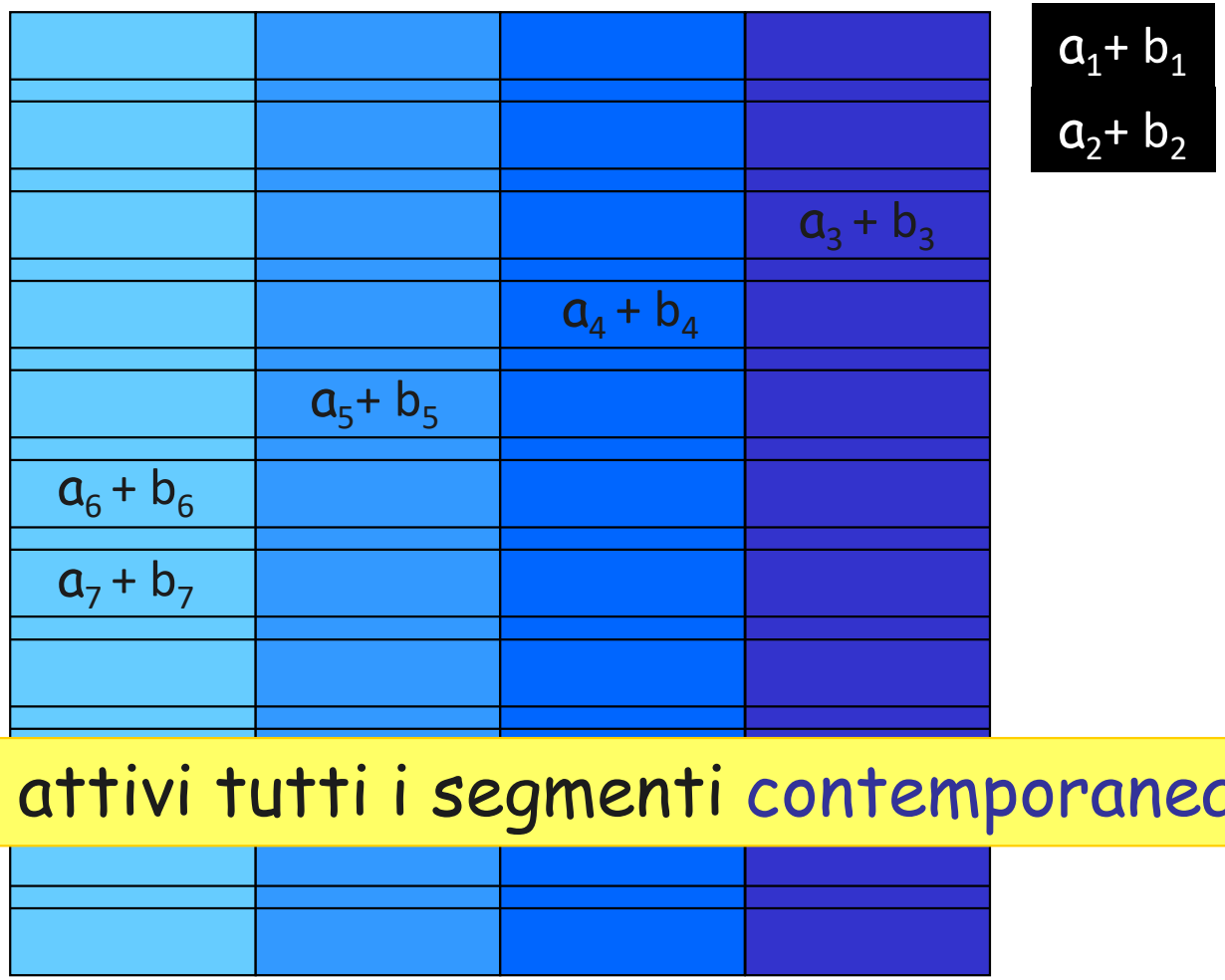
più unità funzionali all'interno di un'unica ALU

Esempio: somma di 100 numeri floating point

$c_i = a_i + b_i \qquad i=1,100$

Unità pipelined

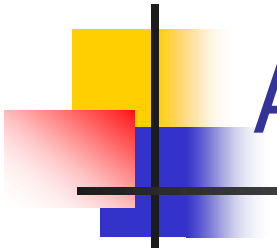
ALU



Hardware:

più unità funzionali all'interno di un'unica ALU

A regime sono attivi tutti i segmenti contemporaneamente



Architetture con unità funzionali *pipelined*

L'uso di unità funzionali pipelined è anche alla base dei

processori vettoriali

Capaci di

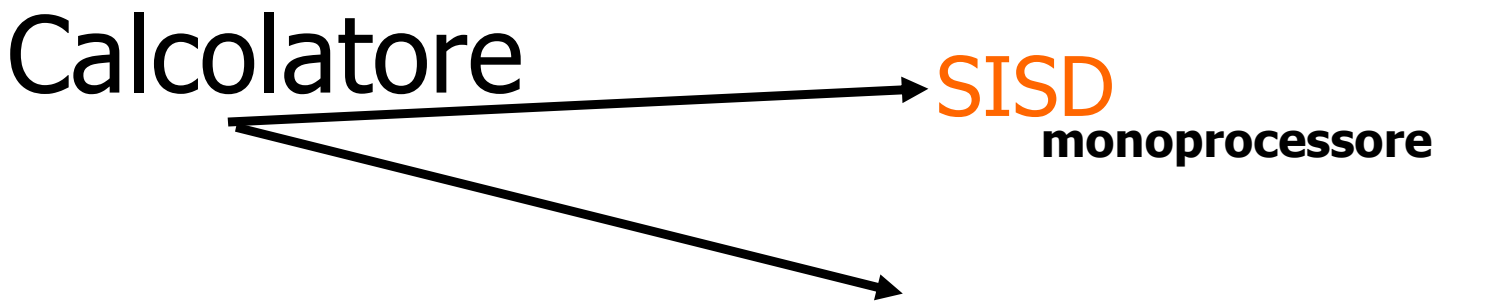
operare efficientemente su dati

strutturati sotto forma di vettori



Attualmente
tutti i microprocessori
utilizzano una struttura pipeline per migliorare le loro prestazioni.

Tassonomia di Flynn (dal 1965)

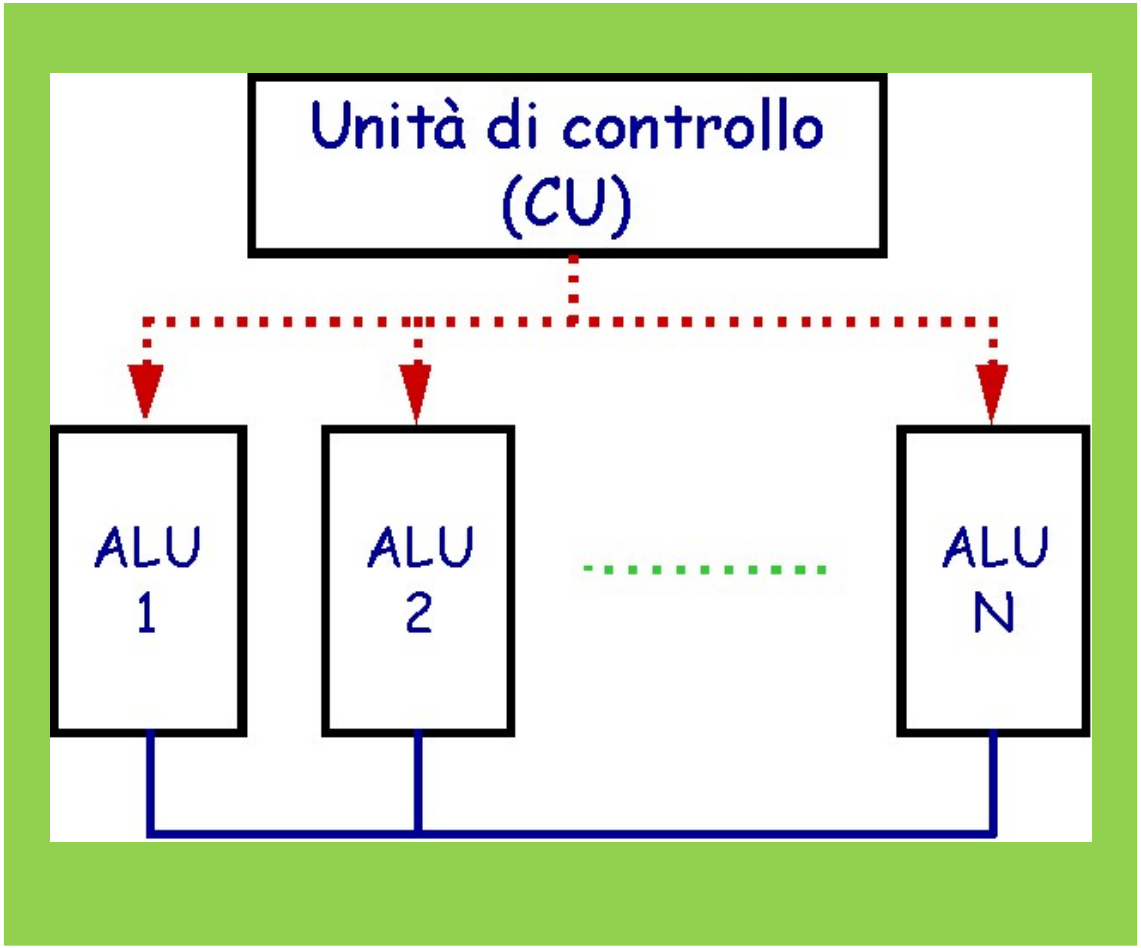


Calcolatori MISD

(Multiple Instruction Single Data)

Parallelismo spaziale

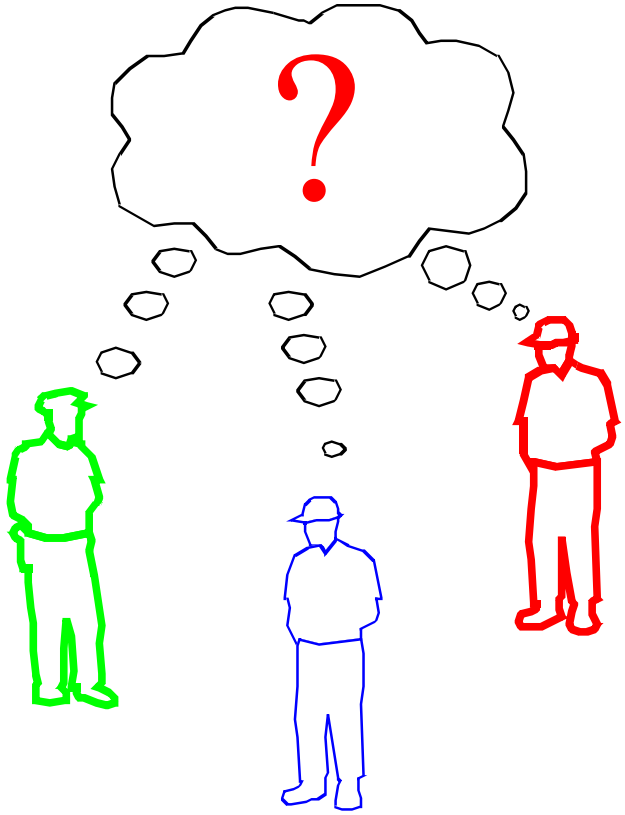
Più unità aritmetico-logiche (ALU) operano sotto un comune controllo (CU) eseguendo in parallelo la ***stessa istruzione*** su ***dati diversi***



Dentro una singola CPU

Esempio: costruzione di una casa





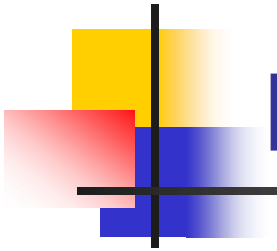
Secondo tipo di parallelismo

Secondo tipo di parallelismo



I tre operai eseguono contemporaneamente
la *stessa azione* su *mattoni diversi*

PARALLELISMO SPAZIALE

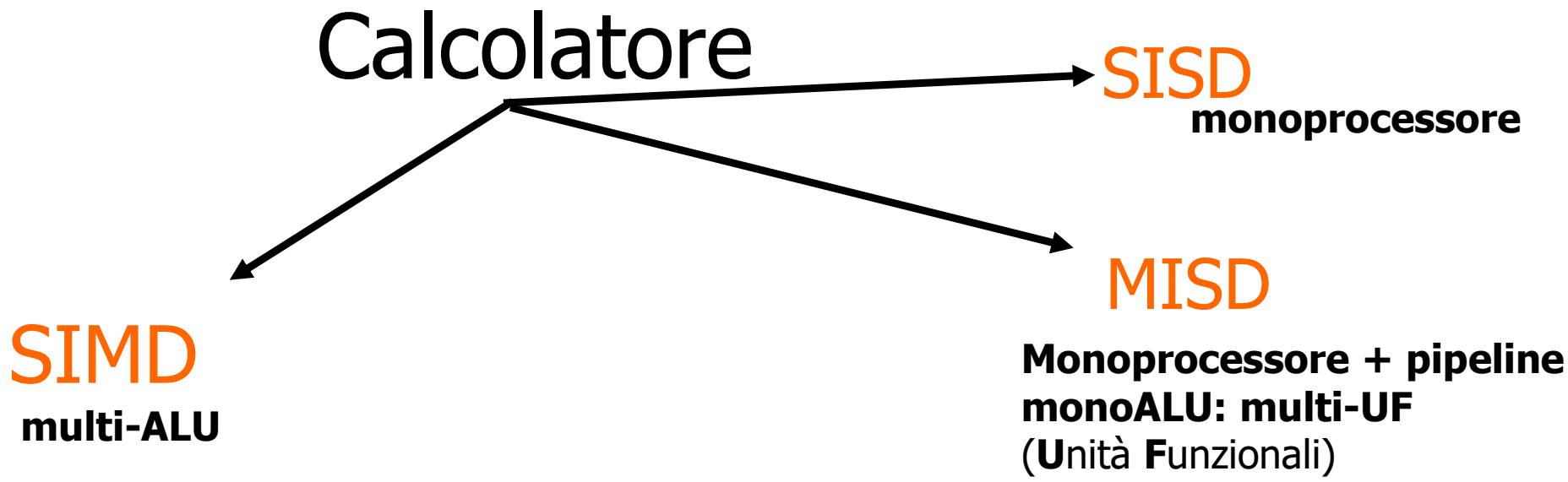


Parallelismo spaziale

Come è realizzato il
parallelismo spaziale in un calcolatore ?

Parallelismo nell'*unità operativa*
(più unità aritmetico-logiche)

Tassonomia di Flynn (dal 1965)



Calcolatori SIMD

(Single Instruction Multiple Data)



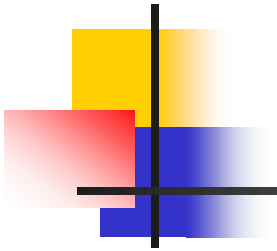
Pausa…?

Terzo tipo di parallelismo



I tre operai eseguono contemporaneamente
azioni diverse su *parti diverse*

PARALLELISMO ASINCRONO



Parallelismo asincrono

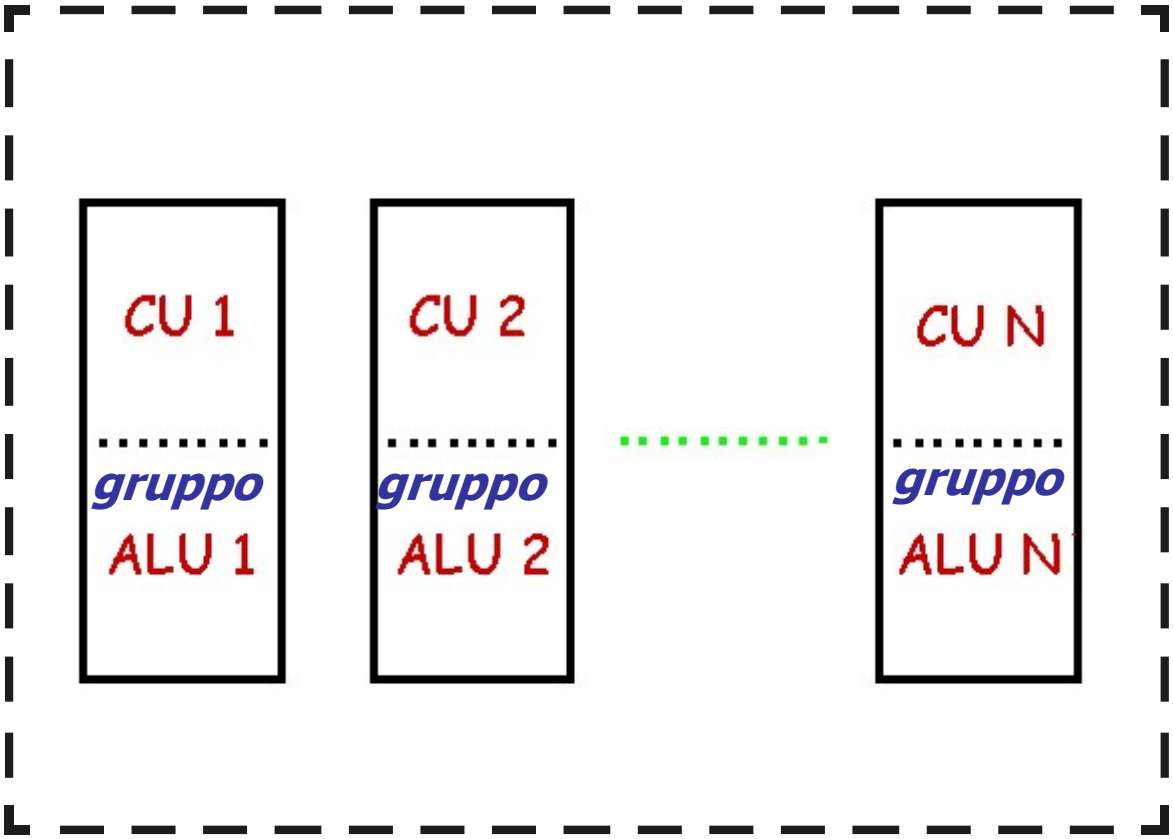
Differenti processori (CPU) cooperano eseguendo istruzioni diverse su dati diversi

Come è realizzato il parallelismo asincrono in un calcolatore ?

Parallelismo delle *CPU*
(più CPU = ALU+CU)

Parallelismo asincrono

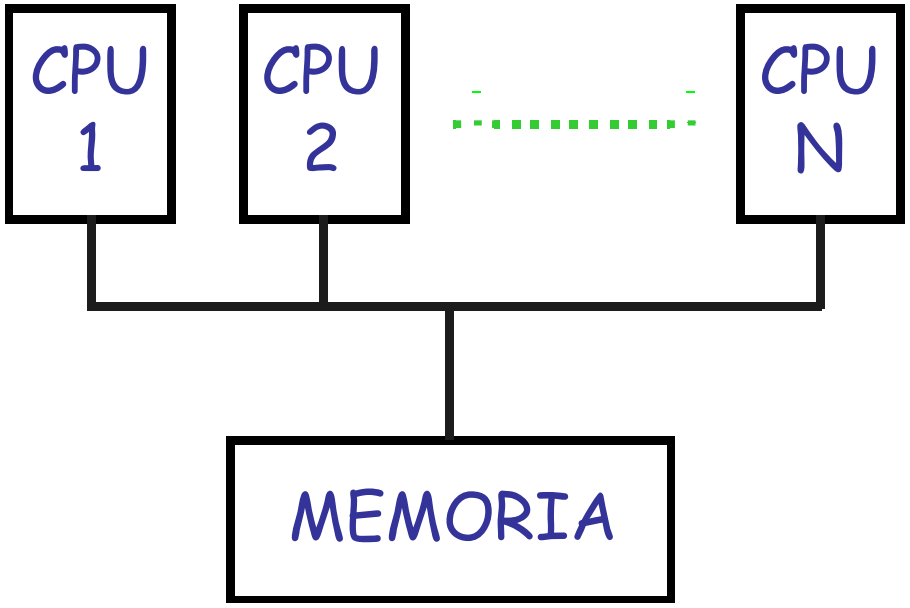
Più CPU (ALU + CU)
eseguono in parallelo
le ***istruzioni diverse***
su ***dati diversi***



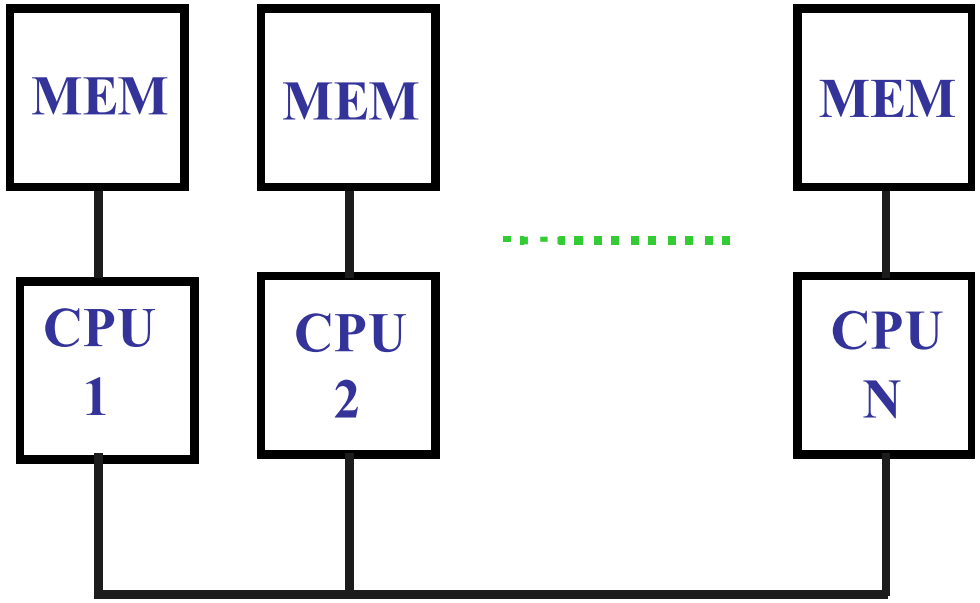
Calcolatori MIMD
(Multiple Instruction Multiple Data)

MIMD

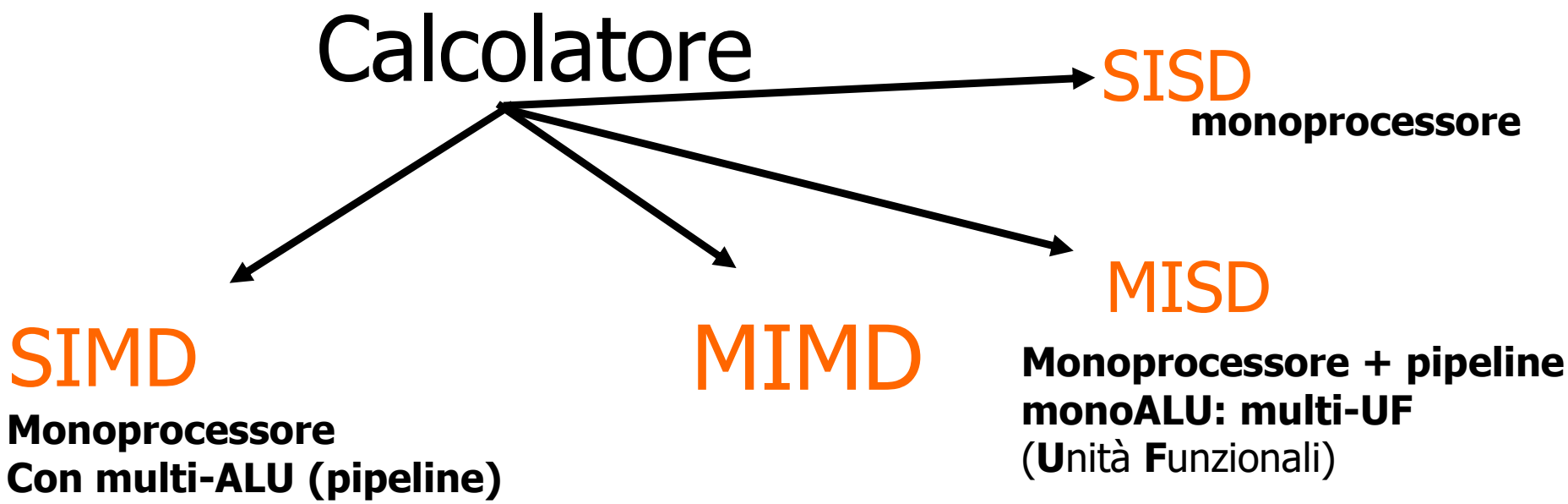
Calcolatori MIMD a memoria condivisa (shared-memory)

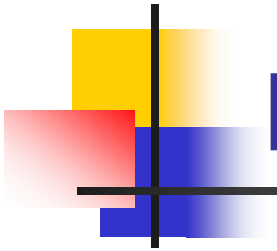


Calcolatori MIMD a memoria distribuita (distributed-memory)



Tassonomia di Flynn (dal 1965)



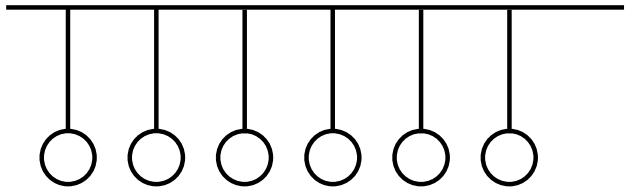


MIMD

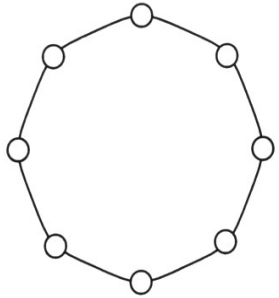
Come sono collegate
CPU e *memorie*
in un calcolatore
MIMD?

MIMD: collegamento CPU-memoria

Calcolatori MIMD a memoria distribuita

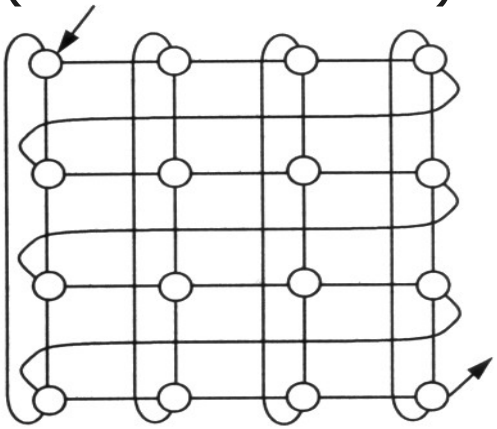


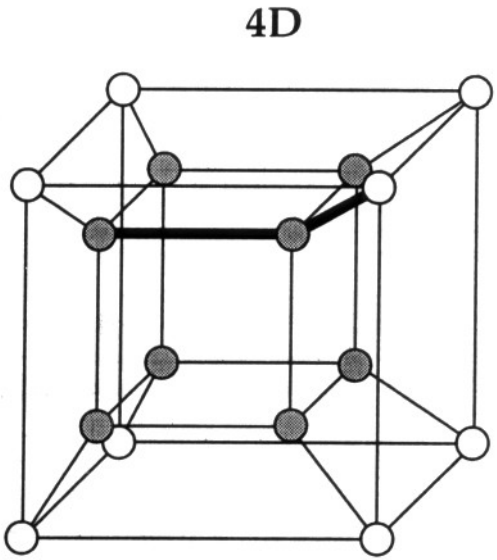
bus



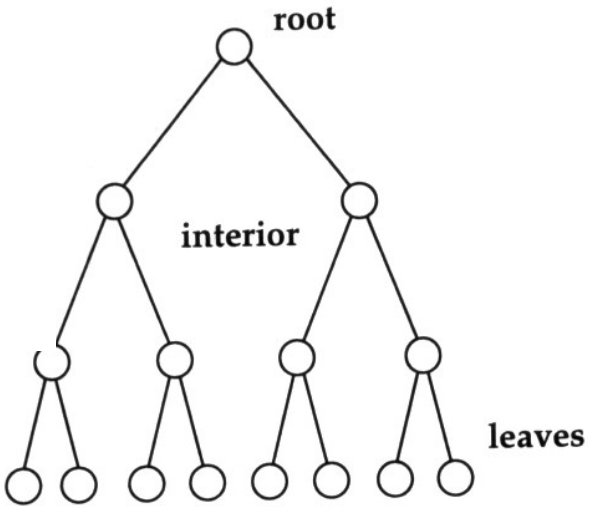
anello

reticolo 2d
(eventualm. *toro*)





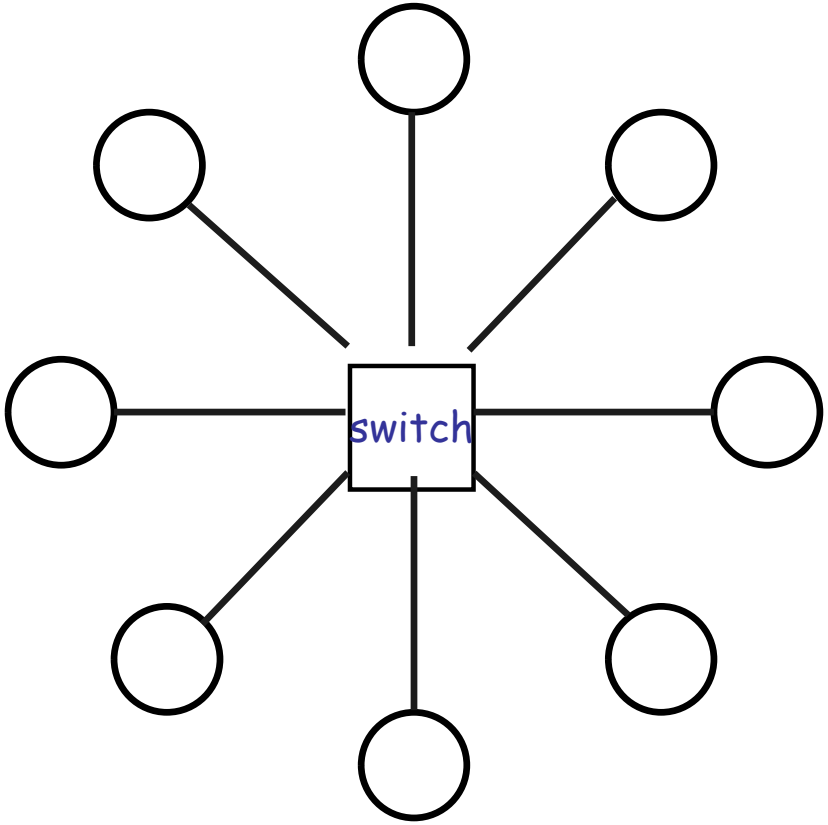
ipercubo



albero

MIMD: collegamento CPU-memoria

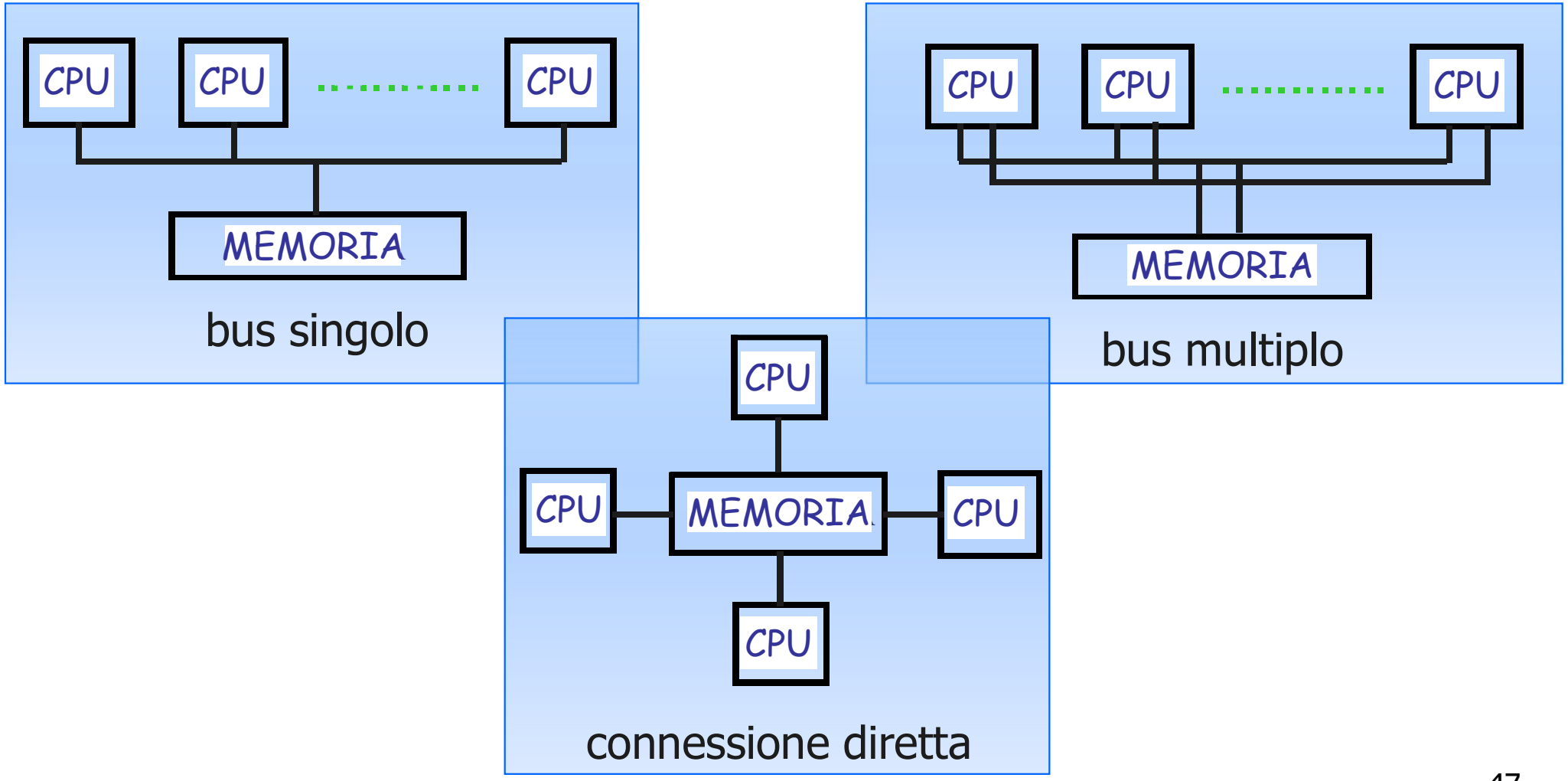
Calcolatori MIMD a memoria distribuita



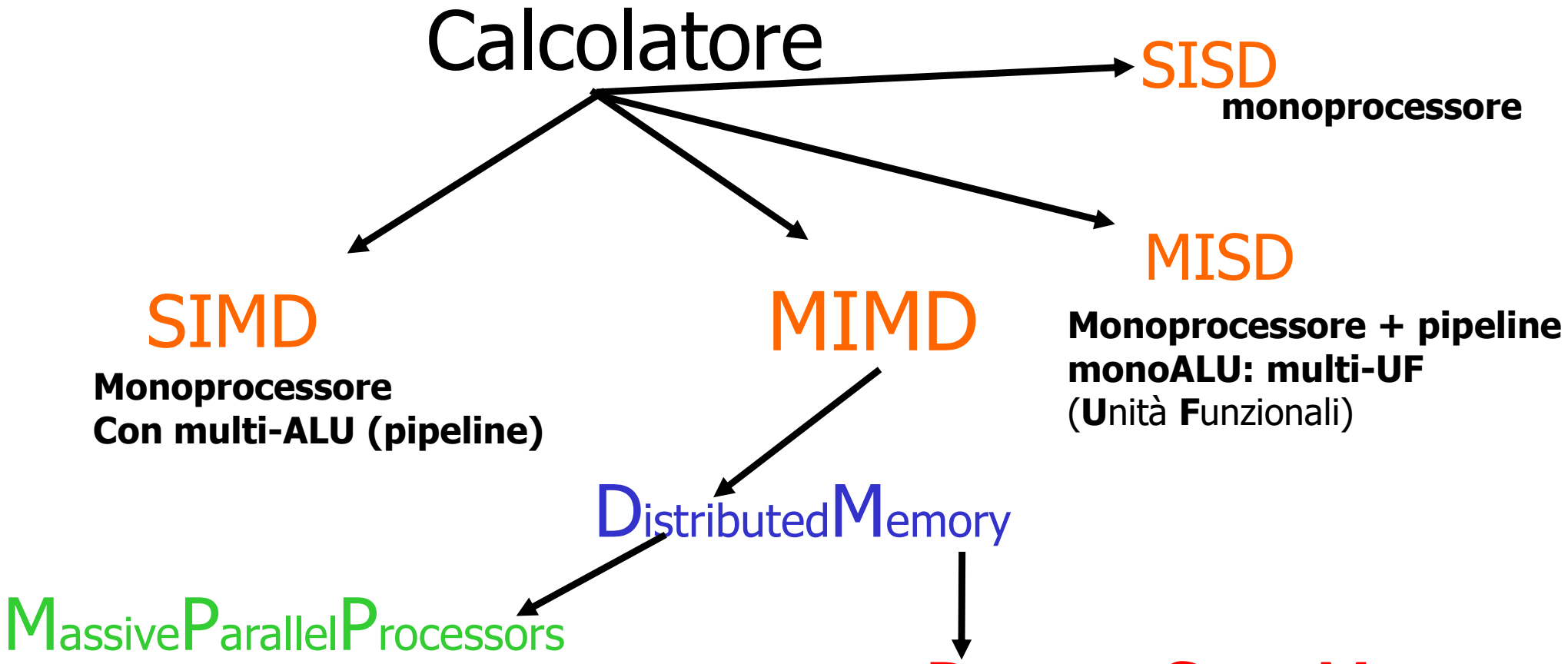
stella

MIMD: collegamento CPU-memoria

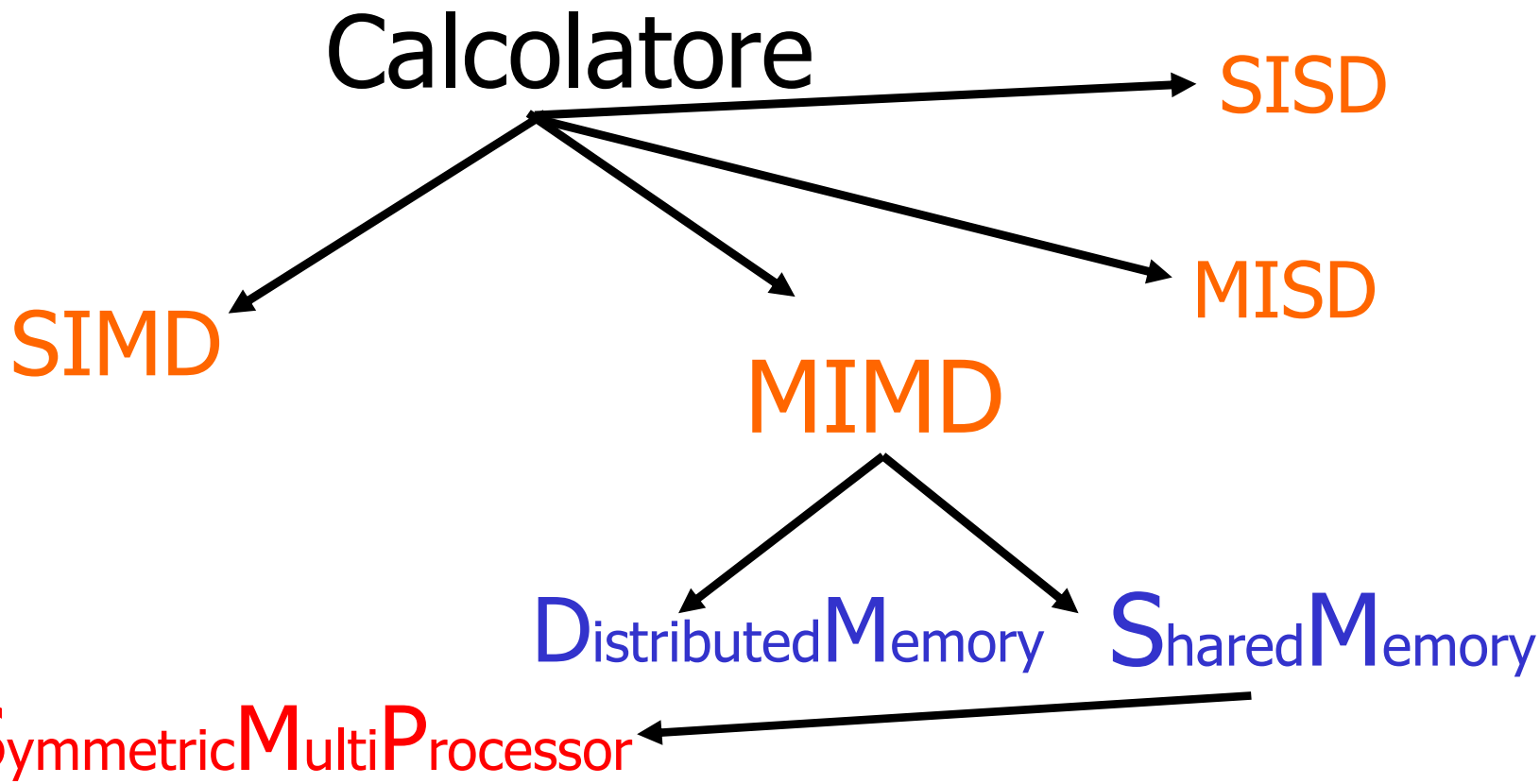
Calcolatori MIMD a memoria condivisa



Tassonomia di Flynn (dal 1965)

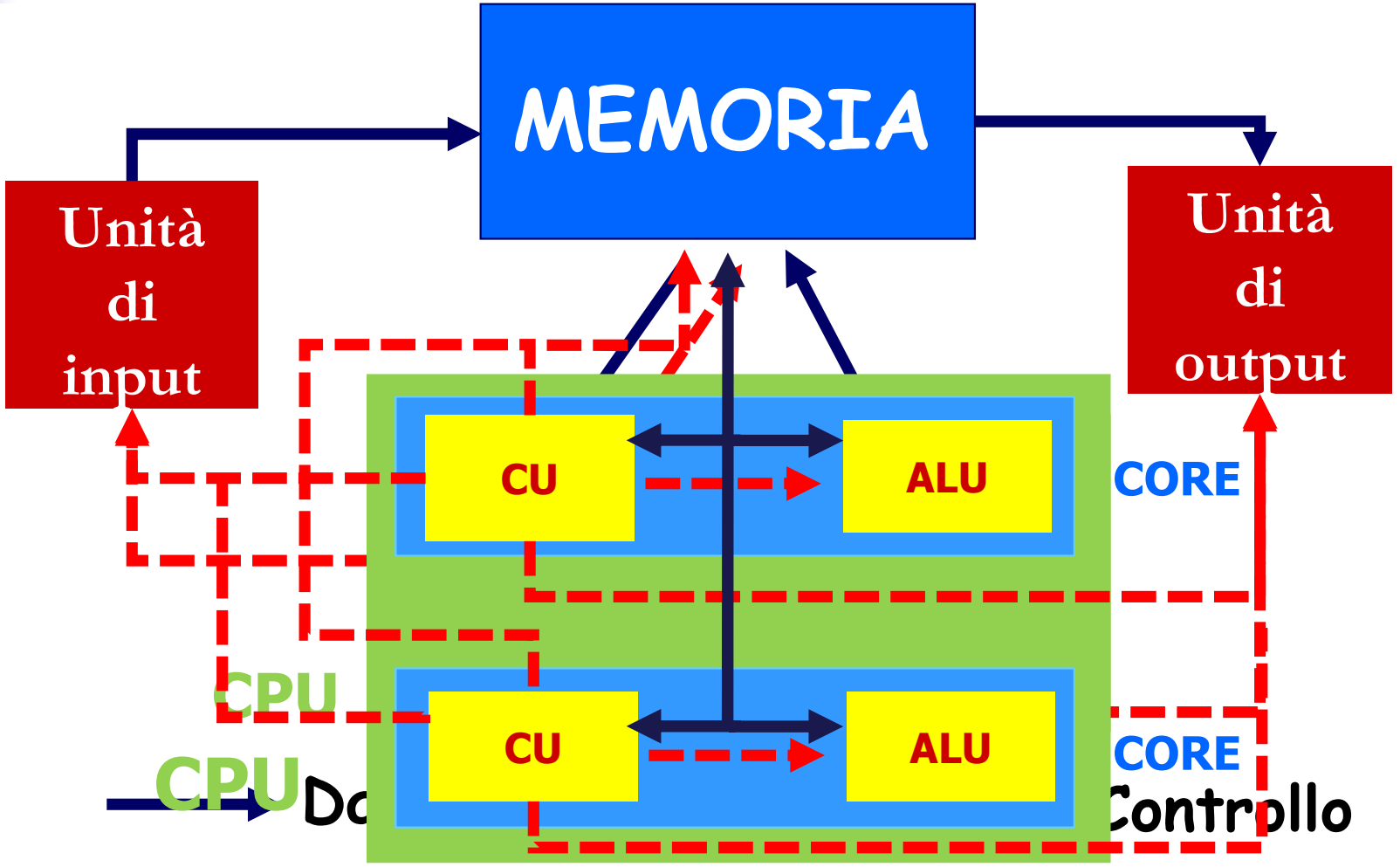


Tassonomia di Flynn (dal 1965)

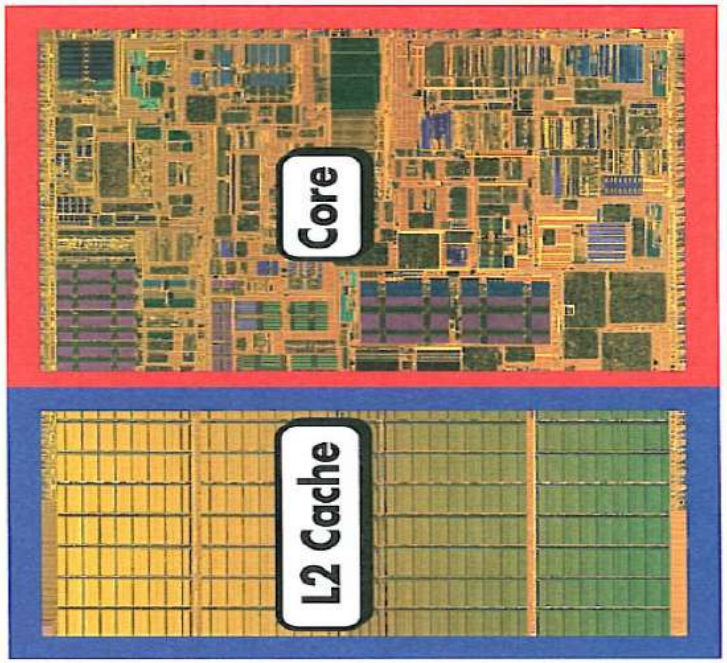


Termine utilizzato per un'architettura in cui due o più micro-processori **identici** sono connessi ad una singola memoria principale condivisa in un'unica macchina.
(**MULTICORE INTEL**)

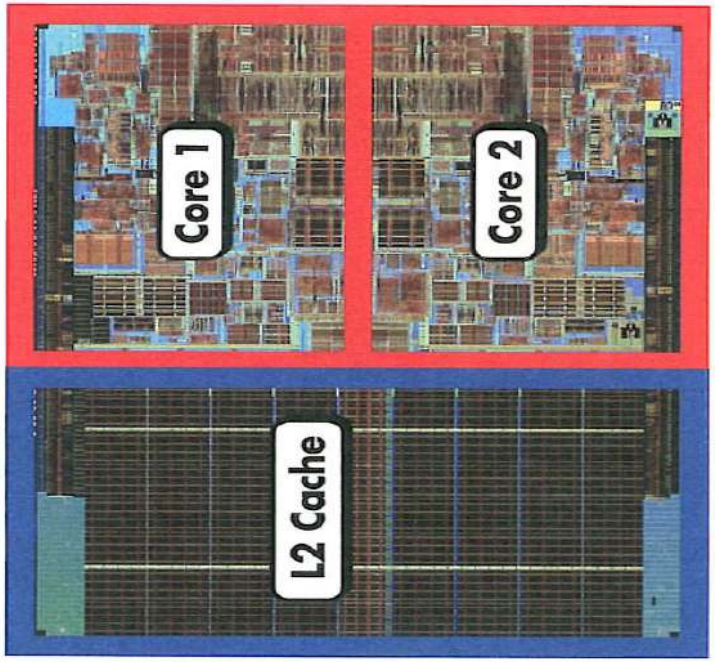
Multicore


MEMORIA
Unità di input
Unità di output
CU
ALU
CORE
CU
ALU
CORE
CPU
CPU
Controllo

Multicore: esempi



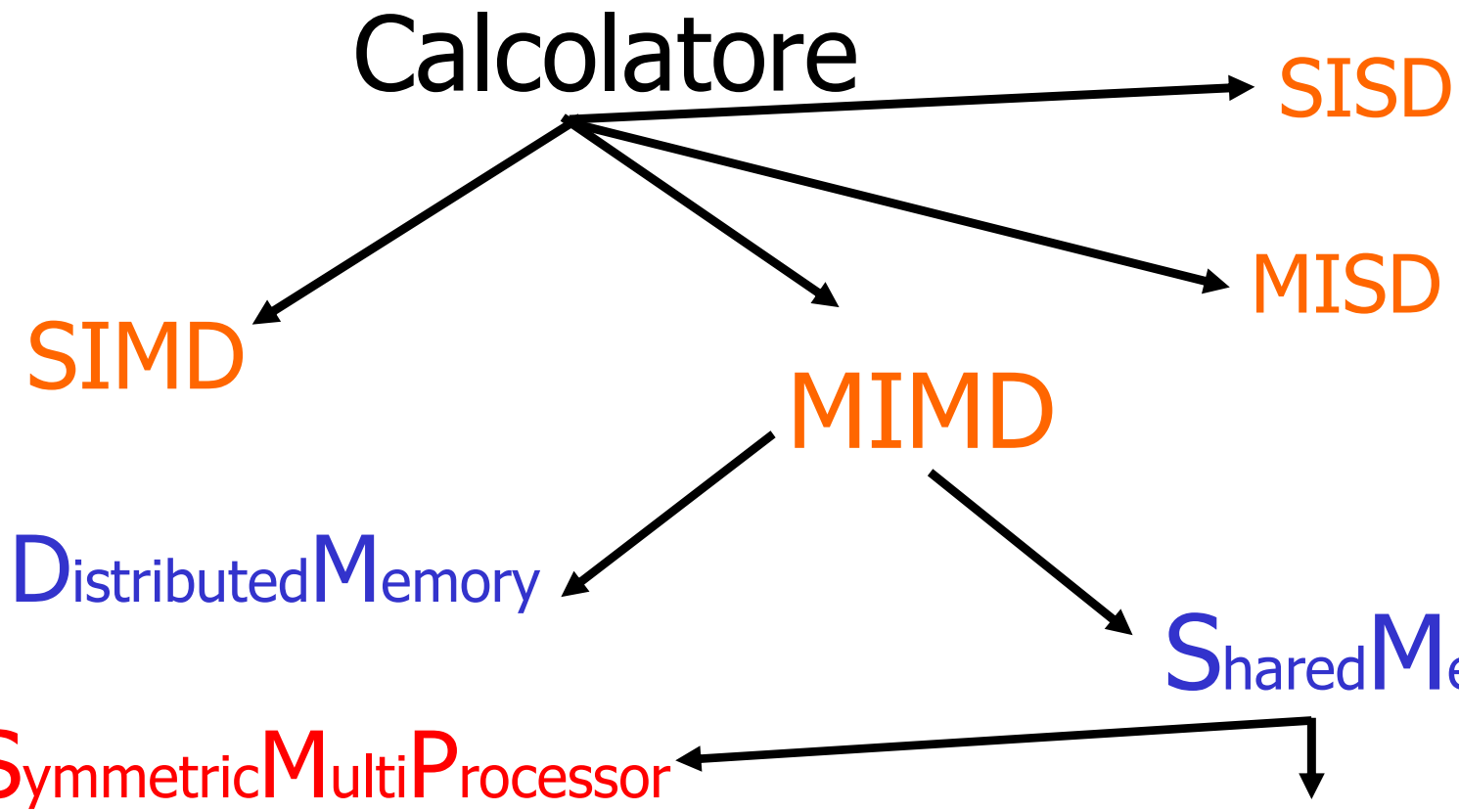
Processore Intel Pentium M (single core)



Processore Intel core duo

Secondo tipo di parallelismo *on-chip*

Tassonomia di Flynn (dal 1965)



Termine utilizzato per un'architettura multiprocessore in cui due o più processori **identici** sono connessi ad una singola memoria principale condivisa. (**MULTICORE INTEL**)

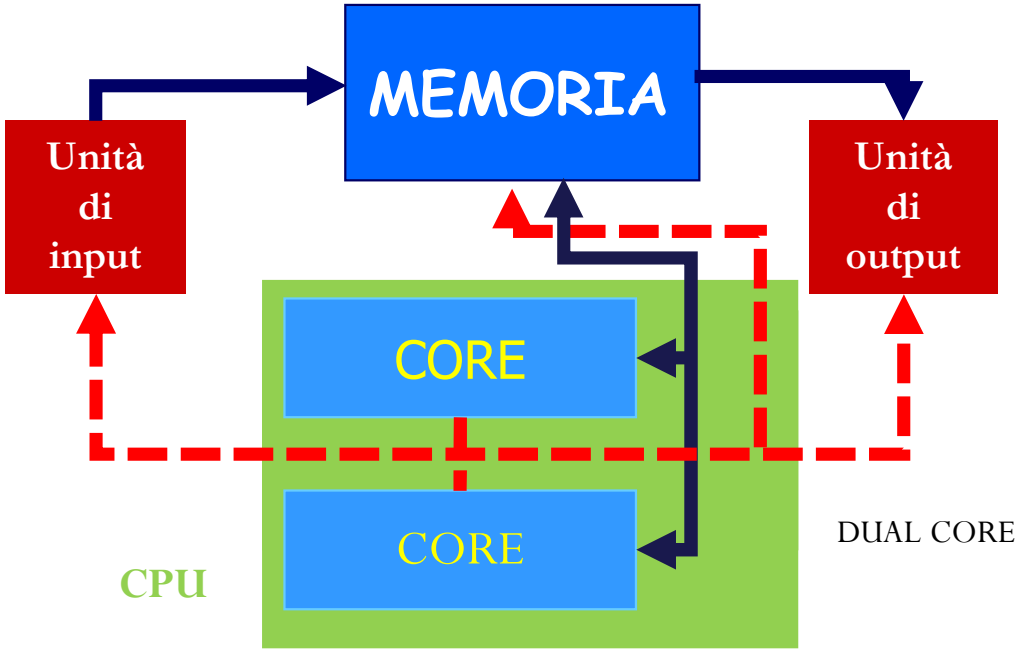
Molte CPU (o unita' processanti) con ruoli differenti in un'unica macchina (**cell IBM, GPU (Graphics Processing Unit), nVidia (TESLA**)

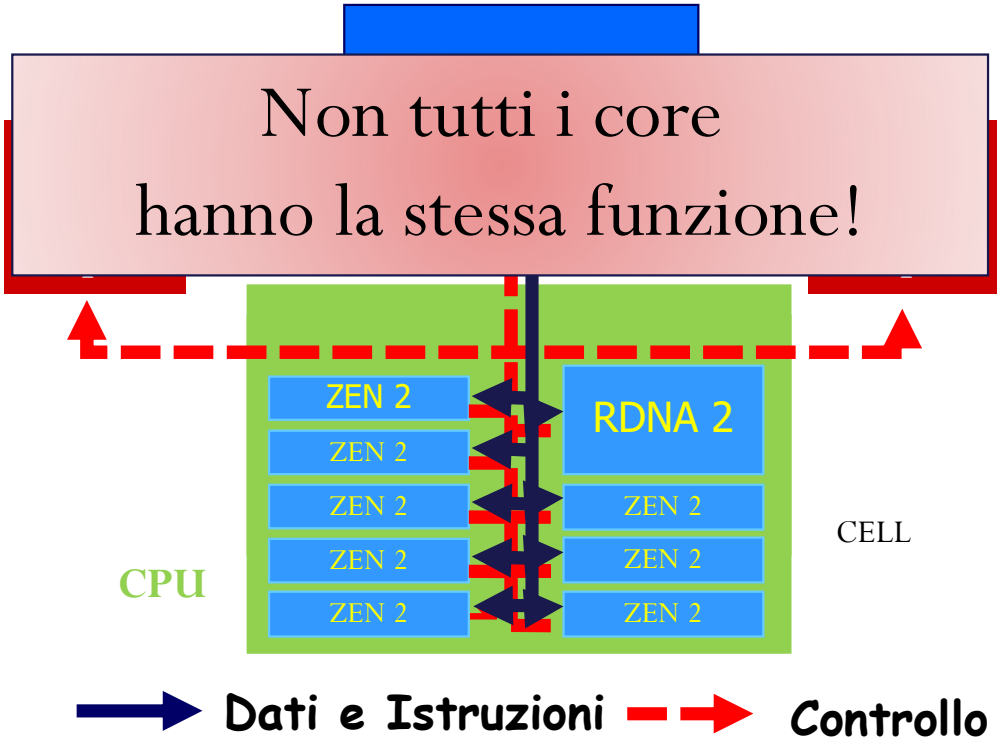
Esempio di calcolatore MIMD-ASMP

Sony (PlayStation 5) novembre 2020



- CPU: **8** Zen 2 Cores at 3.5GHz
- GPU: **1** RDNA 2 at 10.28 TFLOPs, 36 CUs at 2.23GHz





Graphics Processing Units (GPUs)

Dispositivi paralleli che ad oggi si sono guadagnate tanta fama, nascono nel **2000** per l'elaborazione grafica.

La loro architettura si e evoluta molto rapidamente e oggi rappresentano la **più sofisticata forma di macchina parallela.**



Oggi sono usate anche per il genaral purpose **GP-GPU** e sono argomento centrale del corso di High Performance Computing del CdL magistrale in Informatica Applicata (Machine Learning and Big Data)



Attualmente
tutti i microprocessori
utilizzano forme diverse di parallelismo, e nessun sistema in commercio si può definire puramente sequenziale.